FIAP ON

COLHEITA DE DADOS E INSIGHTS - DADOS VALIOSOS E MADUROS

A ELETRÔNICA DE UMA IA

12

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE FIGURAS

Figura 1 - LEDs coloridos 13
Figura 2 - Bateria de 9V alimentando um LED através de um R = 350Ω. O LED está aceso no simulador TinkerCad 15
Figura 3 - ESP32 controlando um LED através de um R = 100Ω (marrom, preto e marrom), porque a saída do ESP32 é de 3,3V 15
Figura 4 - Relé azul da família Arduino/ESP 18
Figura 5 - Relé controlando uma lâmpada simples alimentada no 127V ou 220V 18
Figura 6 - Sensor DHT22 real, com os pinos de alimentação positivo, negativo e a saída de dados out 21
Figura 7 - Pinagem do DHT22 com ou sem plaquinha de suporte 22
Figura 8 - Sensor DHT22 conectado ao ESP32 23
Figura 9 - Sensor ultrassom HC-SR04 27
Figura 10 - Conversor de nível lógico de 5V para 3,3V e vice-versa para adaptar sensores de 5V com o ESP32 29
Figura 11 - Sensor ultrassom detectando distância de um objeto e acionando um LED quando atingir um certo limite 29
Figura 12 - Push buttons de diferentes cores 34
Figura 13 - Circuito com ESP32 com push button no GPIO 22 para acender um LED no GPIO 17 35
Figura 14 - Pinagem do push button quando é pressionado e quando não é pressionado 36
Figura 15 - Sensor PIR de vários ângulos 40
Figura 16 - Detecção do sensor usando uma lente que amplia o campo de visão e o sinal infravermelho detectado 40
Figura 17 - Sensor PIR com a eletrônica disponível no mercado 41
Figura 18 - Sensor PIR conectado no ESP32. LED para ser controlado 41
Figura 19 - Diferentes modelos de buzzer 45
Figura 20 - Conexão de um ESP32 com um buzzer 46
Figura 21 - Desenhos que explicam o servomotor internamente 48
Figura 22 - Servomotor Sg90 Tower Pro 49
Figura 23 - Conexão do servomotor com o ESP32 49
Figura 24 - Sinal PWM com duty próximo dos 90%, isto é, quase toda a onda está em nível On 50
Figura 25 - Sensor LDR “nu” à esquerda; sensor LDR em forma de placa à direita 55
Figura 26 - Circuito ESP32 com sensor LDR em forma de módulo 56
Figura 27 - Sensor MPU6050 59
Figura 28 - Conexão entre ESP32 e o MPU6050 60

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE TABELAS

Tabela 1 - Valores de resistores para diferentes tensões e cores de LED.....14

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 - Código pisca-pisca 16
Código-fonte 2 - Código que liga/desliga o relé.....19
Código-fonte 3 - Código-fonte para ler umidade e temperatura do DHT22.....24
Código-fonte 4 - Exemplo de como usar o sensor de distância ultrassom.....30
Código-fonte 5 - Exemplo de como programar o botão de pressão.....37
Código-fonte 6 - Exemplo de como programar o uso do sensor PIR 42
Código-fonte 7 - Exemplo de buzzer emitindo sons em frequências diferentes.....47
Código-fonte 8 - Exemplo de código para controlar servomotor 52
Código-fonte 9 - Exemplo de como programar o LDR 57
Código-fonte 10 - Exemplo de uso do sensor giroscópio.....61

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

SUMÁRIO

1 A ELETRÔNICA DE UMA IA 7
1.1 Introdução 7
1.1.1 A base física da Inteligência Artificial 7
1.1.2 Arduino: Uma ferramenta prática e versátil 8
1.1.3 Integração com IoT e sistemas inteligentes 8
1.1.4 Preparação para o futuro da automação e robótica 9
2 BIG DATA USANDO A ELETRÔNICA PARA ALIMENTAR A IA 10
2.1 Envio de dados para a nuvem 11
2.2 Armazenamento local e coleta periódica 11
2.3 Processamento distribuído 11
2.4 Alimentando modelos de Inteligência Artificial 11
2.5 Benefícios da integração Arduino com a IA 12
3 CONCEITOS BÁSICOS DE ELETRÔNICA 13
3.1 O LED (diodo emissor de luz) 13
3.2 Relé 16
3.2.1 Pinagem do relé 17
3.2.2 Programando o relé 19
3.2.3 O que fazer com relé em IA? 19
3.2.4 Benefícios e desafios do uso de relés em IA com ESP32 20
3.3 Sensor de temperatura e umidade (DHT22) 21
3.3.1 Pinagem do DHT22 22
3.3.2 Programando o DHT22 22
3.4 Sensor de ultrassônico HC-SR04 27
3.4.1 O processo de medição 28
3.4.2 Pinagem do HC-SR04 28
3.4.3 Programação do ESP32 para usar o HC-SR04 29
3.4.4 Aplicações do HC-SR04 com IA 33
3.5 Push button 34
3.5.1 Pinagem 36
3.5.2 Programação usando Push Button 36
3.5.3 Aplicações do push button em sistemas de IA 38
3.6 Sensor de movimento PIR 39
3.6.1 Funcionamento e pinagem do sensor PIR 39
3.6.2 Programa usando PIR 42
3.6.3 Aplicações do sensor PIR em sistemas de IA 44
3.7 Buzzer e pinagem 44
3.7.1 Programando com o buzzer 46
3.7.2 Aplicações do buzzer em sistemas de IA 47
3.8 Servomotor 48
3.8.1 Funcionamento de um servomotor 50
3.8.2 Programando com servomotor 52
3.8.3 Aplicações do servomotor em sistemas de IA 54
3.9 Sensor de luz LDR 55
3.9.1 Pinagem do LDR 56
3.9.2 Programando com o LDR 57
3.9.3 Aplicação com LDR no mundo da IA 58

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

3.10 Sensor acelerômetro MPU6040 58
3.10.1 Acelerômetro 59
3.10.2 Giroscópio 59
3.10.3 Pinagem do MPU6050 60
3.10.4 Programando com MPU6050 60
3.10.5 Aplicações do MPU6050 em sistemas de IA 64
REFERÊNCIAS 65

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

1 A ELETRÔNICA DE UMA IA

1.1 Introdução

No capítulo anterior, você aprendeu como instalar o ambiente de programação chamado Arduino IDE, como simular projetos de Arduino/ESP usando o Wokwi.com e, ainda, foi introduzido na lógica de programação utilizando C++. Agora, neste capítulo, abordaremos conceitos fundamentais de eletrônica aplicada no contexto de Inteligência Artificial (IA), com foco em sensores e atuadores da família Arduino/ESP.

Você deve estar se perguntando: “Por que estudar eletrônica no mundo da IA?” A eletrônica e a IA estão cada vez mais interligadas no desenvolvimento de tecnologias que estão moldando o futuro.

Com o avanço das áreas de automação, robótica, IoT (Internet das Coisas) e sistemas inteligentes, a integração entre hardware e software se torna essencial. Nesse contexto, estudar eletrônica, especialmente por meio de plataformas acessíveis como a família Arduino, é normal para qualquer pessoa que deseja se destacar no campo da IA. Veja algumas razões:

1.1.1 A base física da Inteligência Artificial

Embora a IA seja frequentemente associada a algoritmos avançados e processamento de dados, em especial, com um Big Data nos bastidores, sua implementação no mundo real depende fortemente de componentes eletrônicos.

Dispositivos eletrônicos como sensores, atuadores, microcontroladores e diversos outros são os elementos que permitem a IA interagir com o ambiente, coletar dados, tomar decisões e realizar ações. Sem uma compreensão razoável da eletrônica, os projetos de IA permaneceriam limitados ao mundo digital, incapazes de causar impacto no mundo físico. Em outras palavras, não teríamos carros inteligentes, robôs auxiliares na limpeza, na recepção de hotéis, em cirurgias, drones, tratores autônomos, porteiro eletrônico etc.

Por exemplo, em um sistema de IA para agricultura de precisão, sensores de umidade do solo, temperatura e luz são utilizados para coletar dados do ambiente. A

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

eletrônica é o que torna possível a coleta desses dados, para que, em seguida, possam ser processados por algoritmos de IA, otimizando o uso de recursos como água e fertilizantes. Nesse sentido, a eletrônica atua como a ponte entre a IA e as suas aplicações práticas.

1.1.2 Arduino: Uma ferramenta prática e versátil

A família Arduino é uma das mais populares e acessíveis plataformas para aprender e aplicar eletrônica, especialmente no contexto de IA. Você aprendeu no capítulo anterior que o Arduino oferece uma interface simples e amigável, permitindo aos iniciantes criar projetos complexos em pouco tempo. Com sua vasta gama de bibliotecas e suporte da comunidade, o Arduino facilita a integração de sensores, atuadores e módulos de comunicação, que são essenciais para sistemas de IA.

Estudar o Arduino é, portanto, uma maneira prática de entender como os sistemas eletrônicos funcionam e como eles podem ser controlados por software. Além disso, o Arduino é amplamente utilizado na prototipagem de projetos de inteligência artificial, permitindo que estudantes e profissionais experimentem novas ideias, realizem testes e implementem soluções inovadoras. Essa capacidade de prototipagem rápida é crucial para o desenvolvimento ágil de sistemas de IA.

1.1.3 Integração com IoT e sistemas inteligentes

A combinação de IA e IoT está criando sistemas cada vez mais inteligentes e conectados. Dispositivos eletrônicos, como os que são controlados por plataformas da família Arduino, podem ser integrados em redes de IoT, onde a IA é utilizada para analisar dados coletados e tomar decisões automáticas.

Estudar eletrônica e Arduino permite entender como esses dispositivos podem ser programados para se comunicar entre si, processar dados localmente ou enviar informações para a nuvem para ter um processamento mais complexo.

Por exemplo, um projeto que usa o Arduino para monitorar a qualidade do ar em uma cidade pode ser combinado com IA para prever níveis de poluição e sugerir ações preventivas. Esse tipo de integração só é possível com uma compreensão clara

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

de como funcionam os componentes eletrônicos, o que reforça a importância do estudo da eletrônica.

1.1.4 Preparação para o futuro da automação e robótica

A robótica e a automação são campos que dependem intensamente da sinergia entre IA e eletrônica. Robôs autônomos, drones, sistemas de manufatura inteligentes e veículos autônomos são alguns exemplos que mostram onde essa integração é crucial.

Aprender a programar microcontroladores como o Arduino/ESP prepara você para enfrentar os desafios do futuro, onde a demanda por profissionais capazes de desenvolver e manter sistemas inteligentes só tende a crescer.

Nesse sentido, o estudo da eletrônica, aliado ao conhecimento em IA, não é apenas uma habilidade técnica, mas também uma preparação estratégica para participar da transformação digital que está redefinindo as indústrias.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

2 BIG DATA USANDO A ELETRÔNICA PARA ALIMENTAR A IA

No mundo contemporâneo, os dados são frequentemente chamados de “o novo petróleo”, pois são a matéria-prima essencial que alimenta muitos dos avanços tecnológicos atuais, especialmente no campo da IA.

A coleta massiva de dados, também conhecida como Big Data, é um pilar para treinar modelos de IA, e com isso, eles aprendem, evoluem e tomam decisões informadas dentro do modelo de negócio.

O Arduino, com sua flexibilidade e capacidade de integrar diversos sensores, desempenha um ótimo papel na coleta desses dados, especialmente em ambientes de IoT e automação.

Já é de conhecimento geral, desde o capítulo anterior, que o Arduino é um microcontrolador acessível e versátil, porém, ainda não discutimos profundamente como ele também pode ser facilmente programado para coletar dados de diferentes tipos de sensores.

Esses sensores podem medir variáveis ambientais como temperatura, umidade, pressão, luminosidade, níveis de qualquer variável, além de dados de movimento, localização, qualidade do ar, entre outros.

A coleta contínua desses dados em larga escala, especialmente quando realizada por múltiplos dispositivos Arduino/ESP distribuídos em diversas localidades de uma propriedade, resulta em um grande volume de informações, caracterizando o que chamamos de Big Data.

Vale mencionar que o Big Data não fica armazenado na memória do Arduino/ESP. Para suportar toda a quantidade de dados que chegam dos sensores, devemos conectar as placas em um banco de dados local ou na nuvem. Por isso é importante entender de SQL e, em seguida, entender o NoSQL.

Para que o Arduino possa participar de um sistema de Big Data, ele precisa ser capaz de transmitir os dados coletados para uma infraestrutura que possa armazenar e processar essas informações em larga escala. Existem várias maneiras de integrar o Arduino com plataformas de Big Data, vejamos a seguir.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

2.1 Envio de dados para a nuvem

O Arduino pode ser configurado para enviar os dados coletados para um servidor na nuvem utilizando protocolos como: MQTT, HTTP, ou FTP. Uma vez na nuvem, os dados podem ser armazenados em bancos de dados escaláveis, como o Amazon S3, Google Cloud Storage, ou Hadoop.

2.2 Armazenamento local e coleta periódica

Em cenários onde a conectividade é limitada, os dados podem ser armazenados localmente em um cartão microSD conectado ao Arduino. Periodicamente, esses dados podem ser coletados manualmente ou transmitidos quando uma conexão estiver disponível.

2.3 Processamento distribuído

Quando múltiplos Arduinos estão coletando dados simultaneamente, o processamento pode ser distribuído utilizando frameworks como Apache Kafka ou Spark. Isso permite que os dados sejam processados em tempo real ou em batch, preparando-os para serem usados por modelos de IA.

Considere que, por exemplo, em uma fazenda inteligente existem vários Arduinos espalhados pelos campos que coletam dados sobre a umidade do solo, temperatura e níveis de nutrientes. Esses dados são enviados para a nuvem, onde são processados e analisados. A grande quantidade de dados coletados (Big Data) permite que os agricultores ajustem a irrigação e a aplicação de fertilizantes com precisão, aumentando a produtividade e reduzindo o desperdício.

2.4 Alimentando modelos de Inteligência Artificial

Uma vez que os dados coletados pelo Arduino são armazenados e processados, eles se tornam “alimento” para modelos de IA. Esses modelos de IA, que podem incluir aprendizado de máquina (Machine Learning), aprendizado profundo

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

(Deep Learning), ou outras técnicas, utilizam os dados para aprender padrões, prever eventos futuros ou tomar decisões automatizadas.

Para treinar um modelo de IA, uma grande quantidade de dados rotulados é necessária. Os dados coletados pelo Arduino são utilizados para treinar o modelo, permitindo que ele aprenda a associar entradas (como dados de sensores) com saídas específicas (como a previsão do clima ou a detecção de anomalias).

Após o treinamento, o modelo de IA pode ser utilizado para fazer inferências em tempo real. Por exemplo, um modelo treinado com dados ambientais coletados por Arduinos pode prever a qualidade do ar em diferentes regiões ou alertar sobre condições meteorológicas adversas.

A coleta contínua de dados permite que os modelos de IA sejam constantemente refinados e atualizados. Novos dados coletados pelo Arduino podem ser usados para ajustar e melhorar a precisão dos modelos ao longo do tempo.

Em uma planta industrial, Arduinos podem monitorar a vibração e a temperatura de máquinas críticas. Esses dados são alimentados em um modelo de IA treinado para prever falhas antes que elas ocorram, assim, o sistema pode alertar os operadores para realizar manutenção preventiva, evitando tempo de inatividade e custos elevados.

2.5 Benefícios da integração Arduino com a IA

O Arduino/ESP, embora pareça um brinquedo, está crescendo muito. Ele atende desde entusiastas até grandes corporações, permitindo que qualquer pessoa possa contribuir para a criação de Big Data. Além disso, com o Arduino/ESP, é fácil escalar a coleta de dados distribuindo múltiplos dispositivos em várias localidades, aumentando a quantidade e qualidade dos dados coletados. Eles também podem ser programados e adaptados para coletar exatamente os dados que são relevantes para um determinado modelo de inteligência artificial, aumentando a eficácia do sistema.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

3 CONCEITOS BÁSICOS DE ELETRÔNICA

A eletrônica é a ciência que estuda o controle do fluxo de elétrons em circuitos, seja em dispositivos simples como LEDs, até em sistemas complexos como microcontroladores e sensores avançados.

Compreender os conceitos básicos da eletrônica é importante para qualquer pessoa que deseje ingressar no mundo da tecnologia, especialmente em áreas interdisciplinares como a IA e IoT. Vejamos a seguir alguns componentes necessários para esse curso.

3.1 O LED (diodo emissor de luz)

Um LED é um componente eletrônico que emite luz quando uma corrente elétrica passa por ele. Ele é polarizado, ou seja, tem um pino chamado anodo (positivo) e um pino chamado catodo (negativo). O LED é muito utilizado para indicar uma informação de forma simples e objetiva, como sinais visuais, indicadores e em sistemas de feedback. A maneira que o algoritmo pisca o LED pode indicar uma informação relevante ou não, tudo depende de quem está projetando o hardware e o software.

Figura 1 - LEDs coloridos
Fonte: Robótica Educacional Brasil (2024)

Um LED é quase sempre ligado em série com um resistor para limitar a corrente que passa pelo seu corpo. Vale mencionar que LED é muito sensível e queima com

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

facilidade quando ultrapassa a tensão limite em seus terminais. Então, é importante que o resistor R atue como um freio da corrente elétrica para não queimar o LED. Em contrapartida, esse resistor R pode se aquecer e “fritar” se o trabalho de frenagem da corrente for além dos limites dos cálculos usando a Lei de Ohm. Via de regra, você pode ser usar os seguintes valores de resistor R para cada situação:

Cor de LED / Tensão de trabalho (V_{LED})	**Tensão de alimentação (V_{CC}) de 3,3V usar R:**	**Tensão de alimentação (V_{CC}) de 5V usar R:**	**Tensão de alimentação (V_{CC}) de 9V usar R:**	**Tensão de alimentação (V_{CC}) de 12V usar R:**
Vermelho / 2V	100Ω	150Ω	350Ω	500Ω
Amarelo / 2V	100Ω	150Ω	350Ω	500Ω
Verde / 3V	Não precisa de R	100Ω	300Ω	450Ω
Azul / 3V	Não precisa de R	100Ω	300Ω	450Ω
Branco / 3,5V	Não precisa de R	80Ω	275Ω	430Ω

Tabela 1 - Valores de resistores para diferentes tensões e cores de LED
Fonte: Elaborado pelo autor (2024)

Observação: esses valores são aproximados e podem variar ± 10%.

Como o curso de IA não exige um estudo aprofundado em eletrônica, podemos pular a questão das contas matemáticas que envolvem a Lei de Ohm. Mas se você está curioso em saber como esses cálculos de R foram feitos, temos:

$$R = (VCC - VLED) / ILED$$

Considerando que qualquer cor de LEDs puxa uma corrente (I_{LED}) média de 20mA, então temos:

$$R = (VCC - VLED) / 0{,}02$$

Substitua o valor de V_{CC} - V_{LED} de acordo com a tabela acima que você vai encontrar os respectivos valores de R. Como exemplo, considere o LED vermelho ligado em 9V.

$R = (9 - 2) / 0{,}02 = 350\ \Omega$ (anéis laranja verde e marrom, conforme mostra a figura a seguir).

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

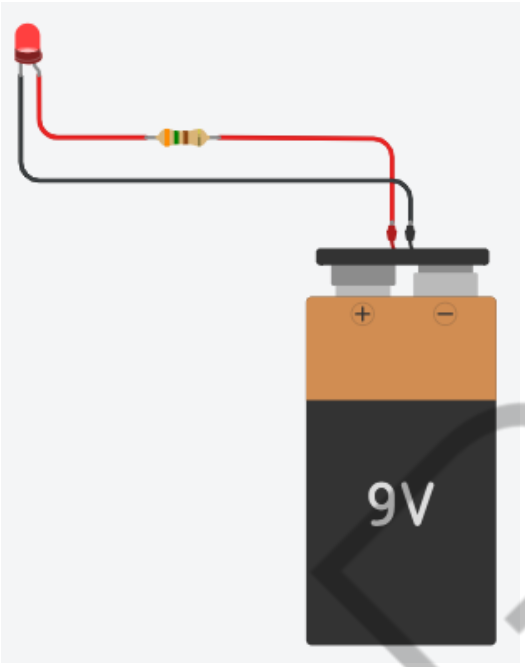

Figura 2 - Bateria de 9V alimentando um LED através de um R = 350Ω. O LED está aceso no simulador TinkerCad
Fonte: Elaborado pelo autor (2024)

Para usar um ESP32 e controlar um LED externo a ele, você pode montar o seguinte circuito no Wokwi.com.

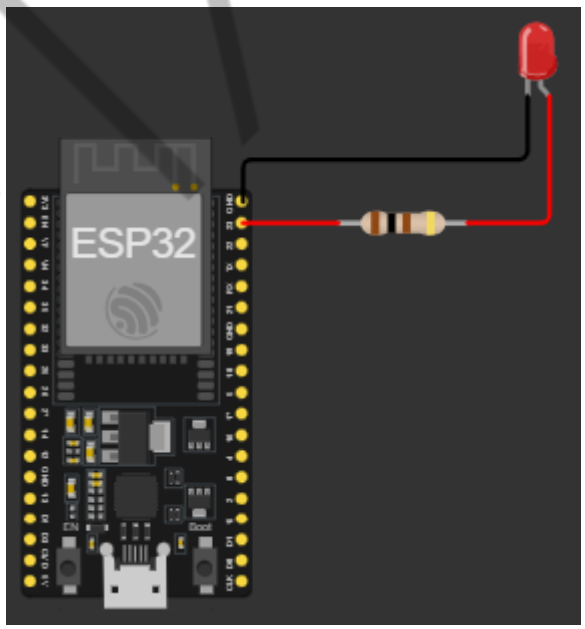

Figura 3 - ESP32 controlando um LED através de um R = 100Ω (marrom, preto e marrom), porque a saída do ESP32 é de 3,3V
Fonte: Elaborado pelo autor (2024)

Para fazer esse LED piscar a cada 1 segundo, podemos aplicar o código a seguir:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
     int ledPin = 23;  // Pino GPIO 23 conectado ao LED
     bool ledState = false;

     void setup() {
         pinMode(ledPin, OUTPUT);  // Define o pino do LED como
saída
     }

     void loop() {
         if (ledState) {
             digitalWrite(ledPin, HIGH);   // Acende o LED com
nível HIGH da tensão
         } else {
             digitalWrite(ledPin, LOW);    // Apaga o LED com
nível LOW da tensão
         }
         delay(1000);  // Aguarda 1 segundo
         ledState = !ledState;  // Alterna o estado do LED. O
símbolo ! indica inversão de valor de uma variável booleana.
     }
```

Código-fonte 1 - Código pisca-pisca
Fonte: Elaborado pelo autor (2024)

Atenção: ao copiar e colar todos os códigos no Wokwi.com, remova os espaços extras marcados por caixinhas no Wokwi.

Note que o if testa se ledState é True. Isso acontece quando não colocamos nenhum operador lógico dentro do if.

3.2 Relé

O relé é um componente eletrônico comum em diversos sistemas de controle e automação, sendo especialmente aplicado em projetos que envolvem microcontroladores como o ESP32. No contexto de um sistema de IA, ele permite integrar sistemas eletrônicos que podem controlar dispositivos de alta potência com segurança e eficiência, ampliando as possibilidades de automação inteligente.

Um relé é basicamente um interruptor de luz, porém eletromecânico, que permite controlar dispositivos “perigosos” que envolvem tensão elétrica da tomada 127V ou 220V, como lâmpadas e motores, usando sinais de baixa potência (como os de um microcontrolador). Ele possui esse formato de caixinha, porque internamente tem uma pequena bobina que forma um eletroímã que atrai um pedaço de metal para gerar um contato em outro metal. Por isso, parece um interruptor residencial simples.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

O relé é muito usado para controlar dispositivos de corrente alternada (AC), embora também seja possível controlar dispositivos de corrente contínua (DC), com um microcontrolador.

3.2.1 Pinagem do relé

O relé utilizado no mundo do ESP32 tem a aparência conforme mostra a figura “Relé azul da família Arduino/ESP”. Ele possui três pinos pretos (VCC, GND e IN), que significam:

- **VCC:** pino que recebe 5V de alimentação para energizar o módulo do relé.
- **GND:** pino Ground ou negativo, que fecha o circuito da energização do módulo. O relé precisa do positivo e do negativo para funcionar.
- **IN:** é o pino que recebe o comando de LIGA/DESLIGA do relé. Quem envia esse comando é o ESP32, e por meio do seu código-fonte, do seu algoritmo, decide a hora de LIGAR ou de DESLIGAR a caixinha azul.

E também possui três pinos azuis (NO, COM e NC), conforme essa descrição:

- **NO:** Normally Open, isso significa que esse pino está nativamente aberto, isto é, a corrente elétrica não passa por ele enquanto ele estiver no estado DESLIGADO porque está fisicamente como um interruptor aberto.
- **COM:** Common, ou seja, esse pino é comum entre os pinos NO e NC.
- **NC:** Normally Closed, isso significa que esse pino está nativamente fechado, isto é, a corrente elétrica vai passar por ele enquanto ele estiver no estado DESLIGADO, porque está fisicamente com um interruptor fechado.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

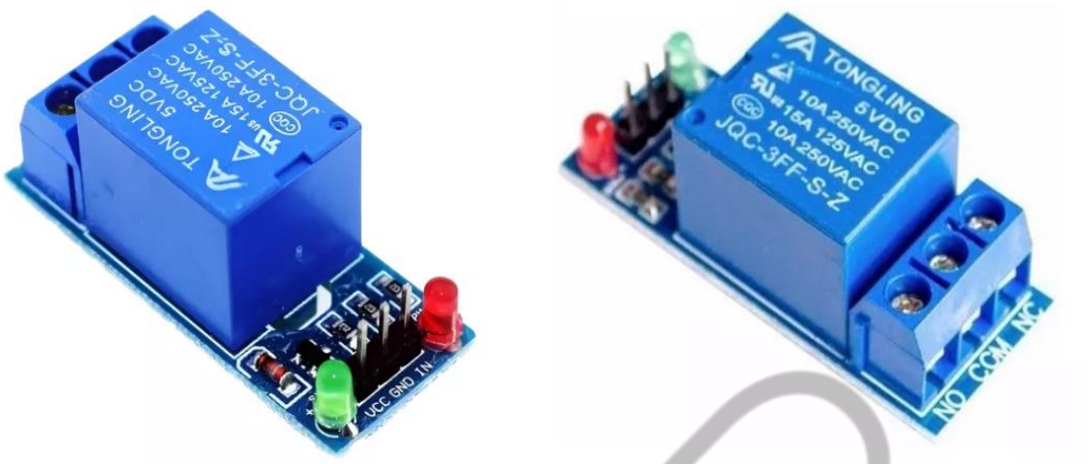

Figura 4 - Relé azul da família Arduino/ESP
Fonte: Robótica Educacional Brasil (2024)

Observação: os três pinos azuis possuem parafusos para fixar o fio na parte de "potência". Os três pinos pretos são VCC, GND e IN.

Quando inserimos um pulso positivo de 3,3V ou 5V no pino IN do relé, temos os pinos COM + NO em contato elétrico físico como se fosse um interruptor simples fechando o circuito da corrente, e assim, a lâmpada se acende.

Quando removemos o pulso do pino IN, isto é, quando conectamos o pino IN ao GND (negativo), os pinos COM + NO desfazem esse contato físico, cessando a corrente, e consequentemente, apagando a lâmpada. Analise a figura "Relé controlando uma lâmpada simples alimentada no 127V ou 220V", que mostra como projeto seria na realidade.

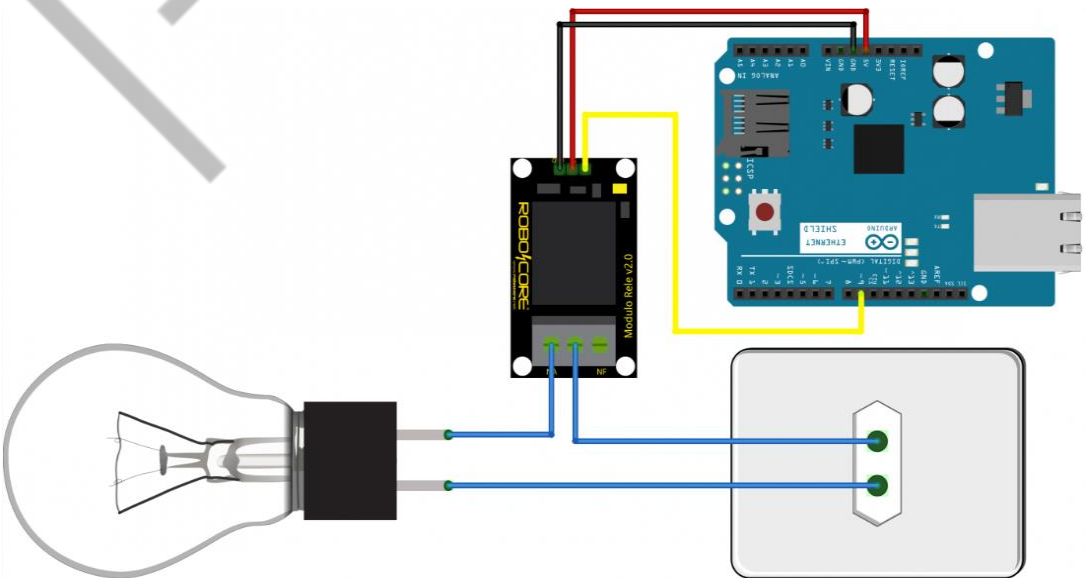

Figura 5 - Relé controlando uma lâmpada simples alimentada no 127V ou 220V
Fonte: Robocore (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Nesse exemplo, o relé está preto, mas sua pinagem é idêntica ao azul.

3.2.2 Programando o relé

O código para o ESP32 ou Arduino para controlar essa lâmpada seria:

```
     int relePin = 9;  // Pino GPIO 9 do Arduino Uno conectado
ao relé ou pino 23 do ESP32
     bool releState = false; //a variável tipo bool só pode
assumir true ou false

     void setup() {
         pinMode(relePin, OUTPUT);  // Define o pino do relé
como saída
     }

     void loop() {
         if (releState) { //testa se releState é True ou False,
isto é, Ligado ou Desligado
             digitalWrite(relePin, HIGH);  // Se o releState for
True, então Liga o relé (ativa a carga)
         } else {
             digitalWrite(relePin, LOW);   // Senão, releState
é False, então desliga o relé (desativa a carga)
         }
         delay(2000);  // Aguarda 2 segundos
         releState = !releState;  // Alterna o estado do relé.
Essa lógica é importante para fazer a lâmpada piscar a cada 2
segundo
     }
```

Código-fonte 2 - Código que liga/desliga o relé
Fonte: Elaborado pelo autor (2024)

Este código utiliza `if-else` para alternar o estado de um relé, controlando assim uma carga externa, como por exemplo, a lâmpada. Contudo, há outras aplicações para o relé no contexto dos sistemas de IA.

3.2.3 O que fazer com relé em IA?

- Em sistemas de casas inteligentes, o ESP32 pode ser utilizado para controlar luzes, ventiladores, ou outros dispositivos elétricos via relés, com base em dados coletados por sensores e analisados por algoritmos de IA.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Por exemplo, a IA pode aprender os padrões de uso de energia de uma casa e automatizar o controle de luzes e eletrodomésticos para economizar energia.

- Em ambientes industriais, o ESP32, combinado com IA, pode monitorar parâmetros como temperatura, pressão, ou vibração, e acionar relés para desligar máquinas ou sistemas automaticamente quando certas condições críticas são detectadas, prevenindo falhas e melhorando a segurança.
- Em aplicações de agricultura, o ESP32 pode controlar sistemas de irrigação ou ventilação em estufas usando relés. A IA pode analisar dados climáticos e do solo para determinar quando e quanto irrigar, otimizando o uso de recursos e aumentando a eficiência da produção.
- Sistemas de segurança inteligentes podem usar o ESP32 para monitorar sensores de movimento ou câmeras, acionando relés para ativar alarmes, trancar portas, ou desligar equipamentos perigosos em caso de intrusão ou outras emergências.

3.2.4 Benefícios e desafios do uso de relés em IA com ESP32

O uso do relé possui os seguintes benefícios:

- Os relés proporcionam um excelente isolamento elétrico entre o circuito de controle de baixa potência (ESP32) e o circuito de alta potência, aumentando a segurança do sistema.
- Relés podem controlar uma ampla gama de dispositivos, desde pequenos motores até grandes sistemas de aquecimento, ventilação e ar condicionado, tornando-os ideais para diversas aplicações de IA.
- Relés, especialmente os eletromecânicos, são conhecidos por sua durabilidade e capacidade de suportar altos níveis de corrente e tensão, tornando-os confiáveis em ambientes industriais e residenciais.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Em relação aos seus desafios:

- Os relés eletromecânicos têm um tempo de resposta mais lento em comparação com soluções eletrônicas puras (como transistores), o que pode ser uma limitação em sistemas que exigem respostas extremamente rápidas.
- O acionamento de relés pode gerar ruído elétrico que interfere em circuitos sensíveis, especialmente em sistemas de comunicação sem fio como o ESP32. Filtragem e proteção adequadas são necessárias para mitigar esses efeitos.
- Em sistemas que exigem múltiplos relés, o espaço e a complexidade do circuito aumentam, exigindo considerações cuidadosas no design do sistema.

3.3 Sensor de temperatura e umidade (DHT22)

O DHT22 é um sensor que mede a temperatura e a umidade. Eles enviam os dados ao microcontrolador via um único pino de dados. Ele é bem útil para monitorar o ambiente, automatizar as residências com base na temperatura e desenvolver projetos de IoT que precise decidir algo com base na temperatura e/ou umidade.

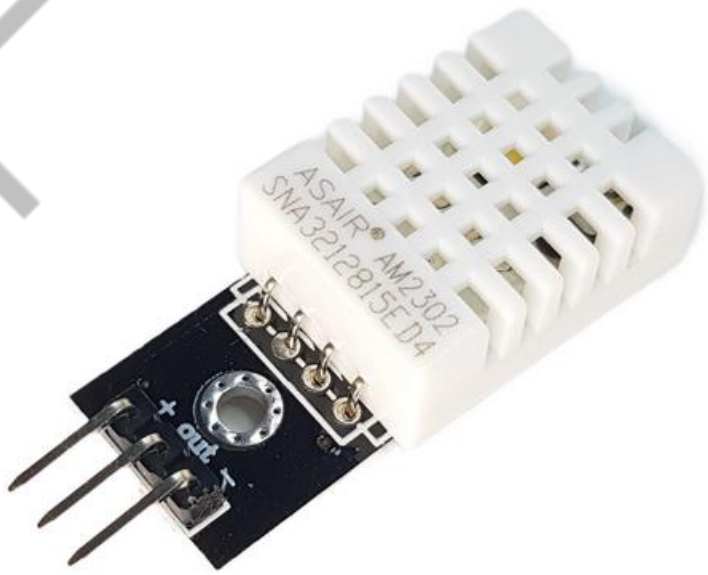

Figura 6 - Sensor DHT22 real, com os pinos de alimentação positivo, negativo e a saída de dados out
Fonte: DIY Ables (2022)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

3.3.1 Pinagem do DHT22

Na figura “Pinagem do DHT22 com ou sem plaquinha de suporte”, tem-se a pinagem do sensor de temperatura e umidade. Note que um dos pinos nunca é utilizado caso você adquira o sensor DHT sem uma plaquinha de suporte. Caso você tenha o sensor com uma plaquinha de suporte, o pino sem uso já está indisponível.

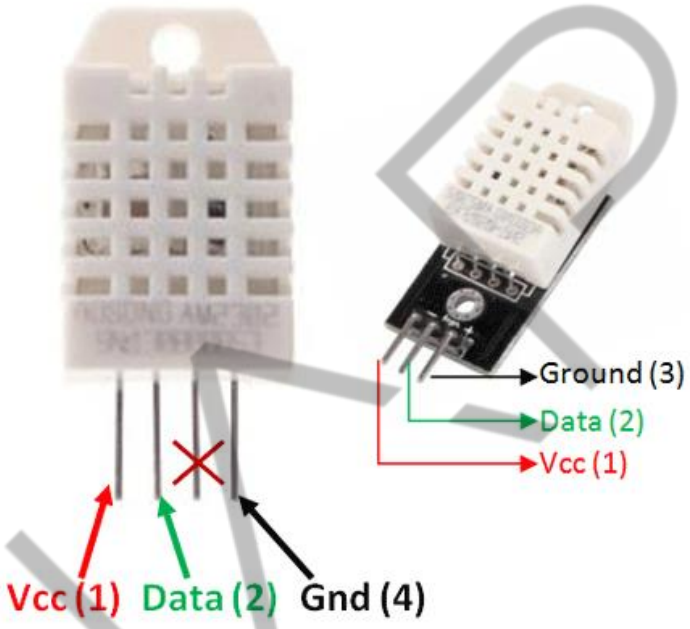

Figura 7 - Pinagem do DHT22 com ou sem plaquinha de suporte
Fonte: Components101 (2018)

3.3.2 Programando o DHT22

O circuito ilustrado na figura “Sensor DHT22 conectado ao ESP32” foi desenvolvido no Wokwi.com, nosso simulador oficial e gratuito do ESP32 e seus sensores. Note que o sensor está ligado no pino +5V do lado esquerdo do ESP32. Isso significa que esse sensor não consegue trabalhar com uma tensão baixa de 3,3V. A sua saída (fio amarelo) vai produzir níveis de tensão acima de 3,3V (porém até 5V), mas nesse caso, não é suficiente para danificar a entrada GPIO 23 que ele está conectado.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

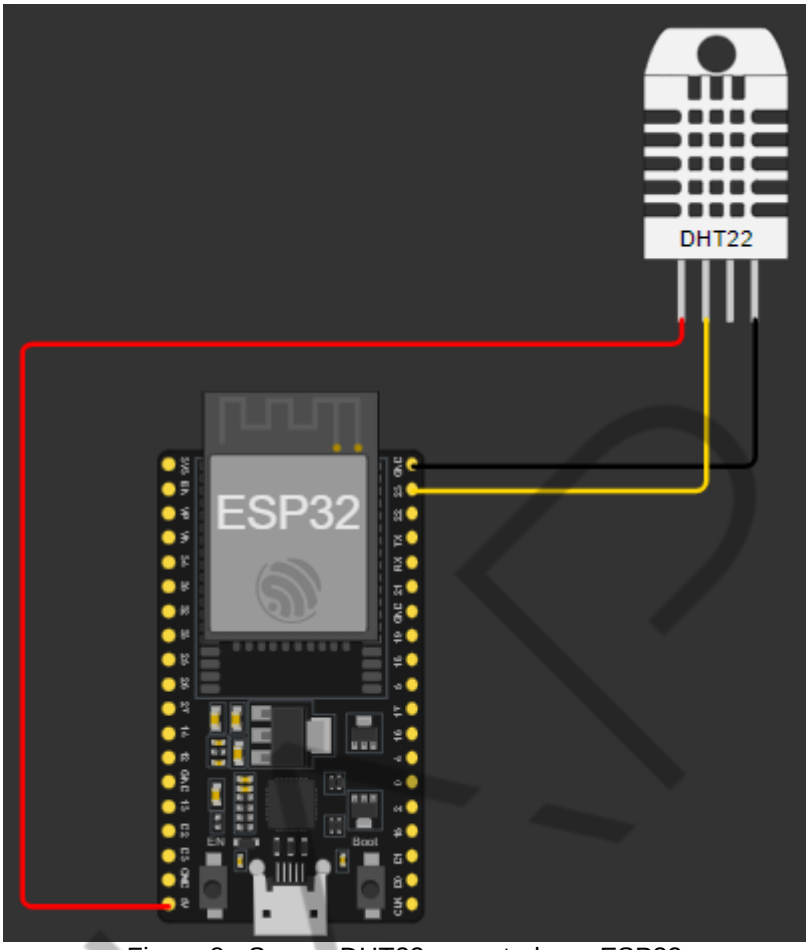

Figura 8 - Sensor DHT22 conectado ao ESP32
Fonte: Elaborado pelo autor (2024)

Nesse circuito, tem o sensor DHT22 conectado no pino de alimentação de 5V no GND (que é o pino de 0V), e tem o pino 23 para ler os dados do sensor.

O código exemplo para a leitura de temperatura e umidade com `if-else`:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
     include <DHT.h>          //precisa incluir essa biblioteca
no seu projeto para que o sensor funcione.

     define pinoDHT 23            //Pino  onde  o  sensor  está
conectado
     define modelo DHT22      //Tipo de sensor DHT22

     DHT dht(pinoDHT, modelo);

     void setup() {
         Serial.begin(9600);
         dht.begin();
     }

     void loop() {
         float h = dht.readHumidity();
         float t = dht.readTemperature();

         if (isnan(h) || isnan(t)) {
             Serial.println("Falha ao ler do sensor DHT!");
             return;
         }

         Serial.print("Umidade: ");
         Serial.print(h);
         Serial.print(" %\t");
         Serial.print("Temperatura: ");
         Serial.print(t);
         Serial.println(" *C");

         if (t > 30.0) {
             Serial.println("Alerta: Temperatura alta!");
         } else {
             Serial.println("Temperatura dentro do normal.");
         }

         delay(2000);         //Aguarda 2 segundos antes de ler
novamente
     }
```

Código-fonte 3 - Código-fonte para ler umidade e temperatura do DHT22
Fonte: Elaborado pelo autor (2024)

Fazendo uma explicação detalhada do código, temos o seguinte:

- **`void setup()`** é uma função que é chamada uma vez quando o programa começa. Ela é usada para inicializar variáveis, definir modos de pinos e iniciar a comunicação serial. Mencionamos isso no capítulo anterior.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- **`Serial.begin(9600);`** inicia a comunicação serial entre o ESP32 (ou outro microcontrolador) e o computador com uma taxa de 9600 bits por segundo (baud rate). Isso permite que o microcontrolador envie dados ao Monitor Serial, onde você pode visualizar as leituras do sensor.
- **`dht.begin();`** inicia o sensor DHT. A função `begin()` é necessária para que o microcontrolador possa se comunicar corretamente com o sensor, preparando-o para começar a fazer leituras de temperatura e umidade.
- **`void loop()`** é uma função que é chamada repetidamente enquanto o microcontrolador está ligado. Além disso, é onde o código principal do programa é executado.
- **`float h = dht.readHumidity();`** lê o valor de umidade atual do sensor DHT e o armazena na variável `h`. A variável é do tipo `float` porque a leitura pode incluir casas decimais, oferecendo maior precisão.
- **`float t = dht.readTemperature();`** lê o valor de temperatura atual do sensor DHT e o armazena na variável `t`. Assim como `h`, `t` é um `float` para armazenar valores com casas decimais.
- **`if (isnan(h) || isnan(t))`** nessa linha, o código verifica se as leituras de umidade (`h`) ou temperatura (`t`) resultaram em `NaN` (Not a Number). Isso pode acontecer se o sensor DHT não estiver conectado corretamente ou se houver um erro na leitura dos dados.
- **`isnan()`** é uma função que retorna `true` se o valor passado para ela for `NaN`. O `NaN` indica a falha na leitura e que o valor obtido não é numérico.
- **`Serial.println("Falha ao ler do sensor DHT!");`** se uma das leituras for `NaN`, esta linha envia uma mensagem ao Monitor Serial indicando que houve uma falha ao ler do sensor. Isso ajuda na depuração, permitindo que o usuário saiba que algo deu errado.
- **`return;`** este comando encerra a execução da função `loop()` para o ciclo atual, impedindo que o código continue a executar com valores inválidos. A função `loop()` será chamada novamente no próximo ciclo, tentando ler o sensor novamente.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- **`Serial.print("Umidade: ");`** envia o texto "Umidade: " para o Monitor Serial, preparando o usuário para ver o valor de umidade a seguir.
- **`Serial.print(h);`** envia o valor de umidade lido (`h`) para o Monitor Serial. Como `h` é um `float`, ele pode incluir casas decimais, oferecendo precisão na leitura.
- **`Serial.print(" %\t");`** adiciona o símbolo de porcentagem `%` após o valor de umidade, seguido de um tabulador `\t` para formatar a saída, tornando-a mais legível.
- **`Serial.print("Temperatura: ");`** envia o texto "Temperatura: " para o Monitor Serial.
- **`Serial.print(t);`** envia o valor de temperatura lido (`t`) para o Monitor Serial. Como `t` é um `float`, ele também pode incluir casas decimais.
- **`Serial.println(" *C");`** adiciona o símbolo de graus Celsius `*C` após o valor de temperatura e, em seguida, move o cursor para a próxima linha com `println()`. Isso garante que a próxima linha de saída comece em uma nova linha no Monitor Serial.
- **`if (t > 30.0)`** verifica se a temperatura lida (`t`) é maior que 30,0 °C. Este é o limite estabelecido no código para determinar se a temperatura está alta.
- **`Serial.println("Alerta: Temperatura alta!");`** se a temperatura for maior que 30 °C, esta mensagem é enviada ao Monitor Serial, alertando o usuário sobre a alta temperatura.
- **`else { Serial.println("Temperatura dentro do normal."); }`** se a temperatura não exceder 30 °C, esta mensagem é enviada ao Monitor Serial, indicando que a temperatura está dentro da faixa normal.
- **`delay(2000);`** aguarda 2000 milissegundos (2 segundos) antes de executar novamente o loop. Isso dá tempo para que o sensor estabilize antes da próxima leitura e evita que o código sobrecarregue o microcontrolador com leituras muito frequentes.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

3.4 Sensor de ultrassônico HC-SR04

O sensor ultrassônico HC-SR04 é muito utilizado em projetos que envolvem medição de distância sem contato. Esse sensor é capaz de medir a distância entre ele e um objeto à sua frente, utilizando ondas ultrassônicas. Seu alcance pega de 2 cm até 300 cm (o objeto precisa estar no mínimo 2 cm na frente do sensor e no máximo 300 cm).

No contexto de projetos com o ESP32, o HC-SR04 oferece uma maneira eficaz e acessível de implementar funcionalidades como detecção de obstáculos, medição de níveis de líquidos, detecção de objetos em lixeiras, detecção de vagas de carros em garagens de shopping, sensor de ré de carros, entre outros.

Quando combinado com a IA, esse sensor pode ser utilizado para coletar dados que alimentarão modelos preditivos ou sistemas de tomada de decisão, como em robôs autônomos ou sistemas de segurança inteligentes.

O HC-SR04 funciona emitindo um sinal ultrassônico de alta frequência (inaudível ao ouvido humano) por meio de seu transmissor. Esse sinal viaja pelo ar até encontrar um objeto, e então reflete de volta para o receptor do sensor. O tempo que o sinal leva para ir, bater no objeto e retornar ao sensor é medido, e com base nessa informação, a distância até o objeto é calculada.

A figura “Sensor ultrassom HC-SR04” mostra como é a aparência real do sensor.

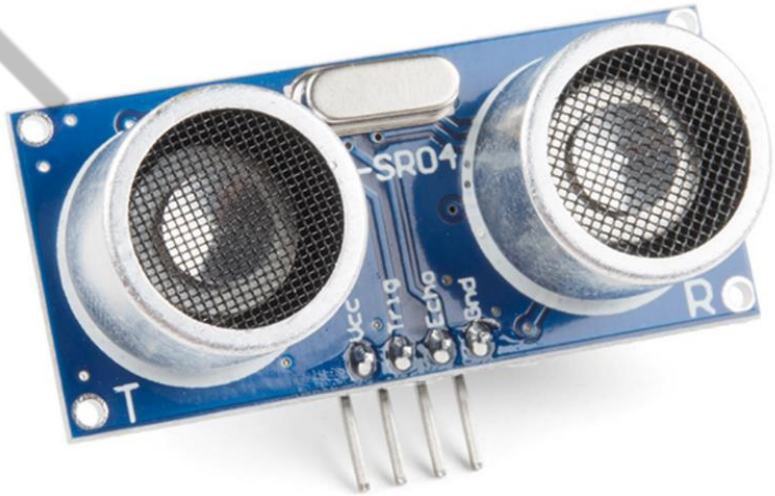

Figura 9 - Sensor ultrassom HC-SR04
Fonte: DFRobot (2023)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Vale mencionar que esse sensor possui um pequeno alto-falante, onde um emite o som inaudível, e o outro, se assemelha a um microfone, que detecta o echo desse pulso.

3.4.1 O processo de medição

a) **Envio do Pulso (Trigger):** o ESP32 envia um pulso elétrico para o pino `Trig` do sensor, fazendo com que ele emita uma onda ultrassônica.
b) **Recepção do Eco (Echo):** quando a onda ultrassônica encontra um objeto, ela é refletida de volta ao sensor e captada pelo pino `Echo`.
c) **Cálculo da Distância:** o intervalo de tempo entre o envio do pulso e a recepção do eco é registrado. Como a velocidade do som no ar é conhecida (aproximadamente 343 metros por segundo a 20 °C), o ESP32 pode calcular a distância ao objeto usando a fórmula:

$$Distância = Tempo\ do\ Pulso \times Velocidade\ do\ Som\ /\ 2$$

Observação: o resultado é dividido por 2 porque o tempo medido é o de ida e volta da onda sonora.

3.4.2 Pinagem do HC-SR04

- **VCC:** alimentação do sensor, geralmente 5V.
- **GND:** terra, conectado ao GND do ESP32.
- **Trig:** pino de Trigger, onde o ESP32 envia um pulso para iniciar a medição. Você pode conectar no pino 5 do ESP32
- **Echo:** pino de Echo, que recebe o pulso de volta refletido pelo objeto. Você pode conectar no pino 18 do ESP32.

Nota: o ESP32 opera com níveis de tensão de 3,3V, enquanto o HC-SR04 geralmente requer 5V para operar. Embora o ESP32 possa tolerar 5V em alguns pinos, é recomendado usar um divisor de tensão ou um conversor de nível lógico para reduzir o sinal de 5V do pino `Echo` para 3,3V.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

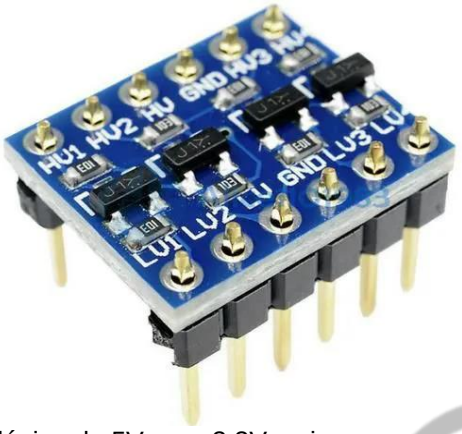

Figura 10 - Conversor de nível lógico de 5V para 3,3V e vice-versa para adaptar sensores de 5V com o ESP32
Fonte: Smartkits (2024)

A figura a seguir mostra como seria o circuito utilizando o sensor ultrassônico para ativar um LED quando atingir uma certa distância do objeto.

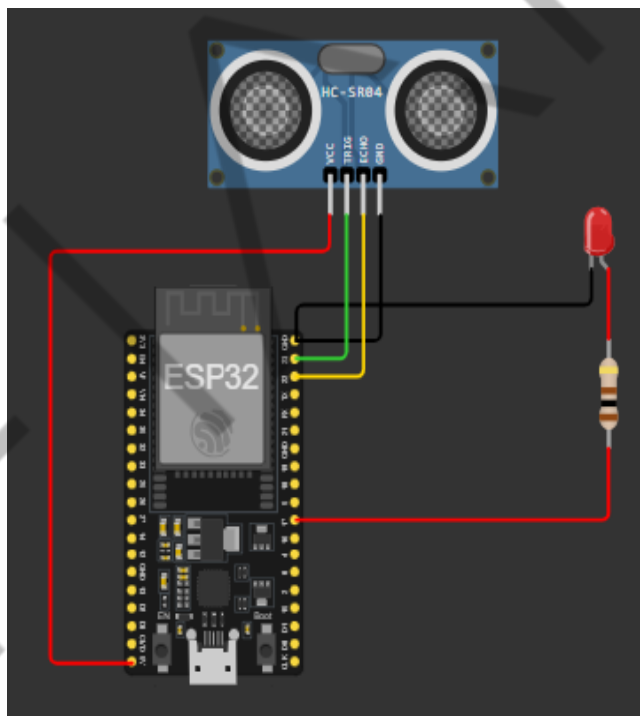

Figura 11 - Sensor ultrassom detectando distância de um objeto e acionando um LED quando atingir um certo limite
Fonte: Elaborado pelo autor (2024)

3.4.3 Programação do ESP32 para usar o HC-SR04

Veremos agora como programar o ESP32 para ler os dados do sensor HC-SR04, calcular a distância e acender um LED se a distância for menor que 20 cm, como um sensor de ré de carro.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
     #define trigPin 23            //  Pino  GPIO  conectado  ao
Trig do HC-SR04
     #define echoPin 22       // Pino GPIO conectado ao Echo do
HC-SR04
     #define pinLED 17        // Pino GPIO conectado ao resistor
de LED

     void setup() {
         Serial.begin(9600);                //   Inicializa   a
comunicação serial
         pinMode(trigPin, OUTPUT);      //  Define  o  pino  Trig
como saída
         pinMode(echoPin, INPUT);       //  Define  o  pino  Echo
como entrada
         pinMode(pinLED, OUTPUT);  // Define o pino do LED como
saída para acender o LED
     }

     void loop() {
         long duration;
         int distance;

         // Envia um pulso de 10us para o Trig
         digitalWrite(trigPin, LOW);
         delayMicroseconds(2);
         digitalWrite(trigPin, HIGH);
         delayMicroseconds(10);
         digitalWrite(trigPin, LOW);

         // Calcula o tempo de duração do pulso de retorno no
Echo
         duration = pulseIn(echoPin, HIGH);

         // Calcula a distância em centímetros
         distance = duration * 0.034 / 2;

         // Exibe a distância no Monitor Serial
         Serial.print("Distância: ");
         Serial.print(distance);
         Serial.println(" cm");

         if(distance                   <                   20){
     digitalWrite(pinLED, HIGH);
         }
         else digitalWrite(pinLED, LOW);

         // Atraso antes da próxima leitura
         delay(500);
     }
```

Código-fonte 4 - Exemplo de como usar o sensor de distância ultrassom
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Explicando o código:

- **`#define trigPin 23`** esta linha define o pino GPIO 23 do ESP32 como `trigPin`. Este é o pino que será conectado ao pino `Trig` do sensor HC-SR04, responsável por enviar o pulso ultrassônico.
- **`#define echoPin 22`** define o pino GPIO 22 do ESP32 como `echoPin`. Este pino será conectado ao pino `Echo` do sensor, que recebe o sinal refletido pelo objeto e calcula a distância. Estas definições facilitam a leitura e modificação do código, pois permitem usar nomes significativos (`trigPin` e `echoPin`) em vez dos números dos pinos diretamente.
- **`void setup()`** é executada uma vez logo após o ESP32 ser ligado ou reiniciado. Aqui, configura-se a comunicação serial e os modos dos pinos.
- **`Serial.begin(9600);`** inicia a comunicação serial a 9600 baud (bits por segundo). Isso permite que o ESP32 envie dados para o Monitor Serial, onde você poderá visualizar as leituras de distância.
- **`pinMode(trigPin, OUTPUT);`** configura o pino `trigPin` (GPIO 23) como uma saída. Este pino será utilizado para enviar o sinal de controle ao sensor HC-SR04.
- **`pinMode(echoPin, INPUT);`** configura o pino `echoPin` (GPIO 22) como uma entrada. Este pino recebe o sinal de retorno do sensor após refletir em um objeto.
- **`pinMode(pinLED, OUTPUT);`** configura o pino do LED como saída e assim, acionará o LED quando necessário.
- **`void loop()`** é uma função que é chamada repetidamente enquanto o ESP32 estiver ligado. É onde o código principal é executado para medir a distância e exibir os resultados.
- **`long duration;`** declara uma variável `duration` do tipo `long` para armazenar o tempo que o sinal leva para voltar ao sensor (medido em microssegundos).
- **`int distance;`** declara uma variável `distance` do tipo `int` para armazenar a distância calculada (em centímetros).

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

A partir daqui, teremos uma sequência de Pulso para iniciar a medição de distância, o ESP32 precisa enviar um pulso elétrico para o pino `Trig` do sensor:

- **`digitalWrite(trigPin, LOW);`** primeiro, garante que o `Trig` esteja em nível baixo (LOW).
- **`delayMicroseconds(2);`** espera por 2 microssegundos. Essa pequena pausa é necessária para garantir a precisão do pulso.
- **`digitalWrite(trigPin, HIGH);`** envia um pulso de 10 microssegundos ao `Trig` para acionar o sensor.
- **`delayMicroseconds(10);`** mantém o `Trig` em nível alto (HIGH) por 10 microssegundos, o tempo necessário para que o sensor emita a onda ultrassônica.
- **`digitalWrite(trigPin, LOW);`** define o `Trig` de volta para nível baixo, completando o ciclo de envio do pulso.
- **`duration = pulseIn(echoPin, HIGH);`** a função `pulseIn()` mede o tempo que o pino `Echo` (GPIO 18) permanece em nível alto (HIGH), que é o tempo que a onda ultrassônica leva para ir até o objeto e voltar ao sensor. Esse valor é armazenado na variável `duration`.
- **`distance = duration * 0.034 / 2;`** esta linha calcula a distância até o objeto usando a fórmula baseada na velocidade do som no ar (~343 metros por segundo ou 0,034 cm/microsegundo). Como o tempo medido é o de ida e volta, dividimos o resultado por 2 para obter a distância real do sensor ao objeto.
- **`Serial.print("Distância: ");`** exibe a palavra "Distância: " no Monitor Serial, preparando o usuário para ver o valor da distância medido.
- **`Serial.print(distance);`** exibe o valor da distância calculada em centímetros.
- **`Serial.println(" cm");`** exibe a unidade de medida (centímetros) e move o cursor do Monitor Serial para a linha seguinte. Utilizar `println` em vez de `print` faz com que a próxima mensagem comece em uma nova linha.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- **`if(distance < 20){`** testa se a distância é menor que 20 cm e aciona um LED.
- **`delay(500);`** introduz um atraso de 500 milissegundos (meio segundo) antes da próxima execução do loop. Esse atraso evita que o sensor seja acionado muito rapidamente, garantindo leituras estáveis e evitando sobrecarregar o microcontrolador.

3.4.4 Aplicações do HC-SR04 com IA

O sensor HC-SR04, quando integrado a um sistema de IA, permite a criação de soluções que requerem a detecção com razoável precisão em distâncias ou de objetos. Por exemplo:

- **Navegação de robôs:** em robótica, o HC-SR04 pode ser usado para detectar obstáculos e ajudar o robô a navegar de forma autônoma, evitando colisões. O ESP32 coleta os dados do sensor, e a IA processa essas informações para decidir a melhor rota a seguir. Exemplo disso são robôs residenciais que varrem o chão.
- **Monitoramento de perímetro:** em sistemas de segurança, o HC-SR04 pode ser utilizado para monitorar perímetros, detectando a presença de pessoas ou objetos em áreas restritas. A IA pode analisar os padrões de movimento e diferenciar entre ameaças reais e falsas.
- **Controle de iluminação:** o sensor pode ser utilizado para detectar quando alguém entra em uma sala, acionando automaticamente as luzes. Com a integração de IA, o sistema pode aprender padrões de uso e ajustar a iluminação com base nas preferências do usuário ou na hora do dia.
- **Medição de níveis em tanques:** o HC-SR04 pode medir o nível de líquidos em tanques. A IA pode prever quando o tanque precisa ser reabastecido com base nos padrões de uso anteriores, otimizando o gerenciamento de recursos.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Portanto, o sensor ultrassônico HC-SR04 é uma ferramenta versátil quando usado com o ESP32, oferecendo uma solução acessível para medir distâncias em uma variedade de aplicações. Quando combinado com IA, ele permite a criação de sistemas inteligentes que podem interagir com o ambiente de forma autônoma e eficiente. Seja em robótica, automação residencial, ou em sistemas de segurança, o HC-SR04 demonstra sua versatilidade e importância como componente em projetos modernos.

3.5 Push button

Um push button é um dos componentes eletrônicos mais básicos em projetos de eletrônica. Ele serve como uma entrada digital simples que pode ser usada para acionar diferentes ações em um sistema. Observe a imagem real de um push button apresentada na figura “Push buttons de diferentes cores”.

Figura 12 - Push buttons de diferentes cores
Fonte: Bitmarker (2024)

Note que cada botão possui quatro pinos, porém você usa dois do mesmo lado, seja direito ou esquerdo, como entrada de informação no ESP32.

No contexto do ESP32, um push button permite que o microcontrolador receba sinais de entrada de um usuário ou de outro sistema, podendo ser utilizados para desencadear funções específicas, como ligar ou desligar um dispositivo, iniciar uma sequência de eventos, ou até mesmo enviar comandos para um sistema mais complexo, como um modelo de IA.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

O circuito a seguir foi montado no Wokwi. Observe que no mesmo pino GND, há mais de uma conexão, sendo que uma está indo para o LED e a outra está indo para o push button. Isso é perfeitamente possível. Você pode ligar quantos fios quiser em um mesmo pino. O comum é usar os pinos GND e 3.3V para espalhar positivos e negativos entre os componentes eletrônicos que circundam o ESP32.

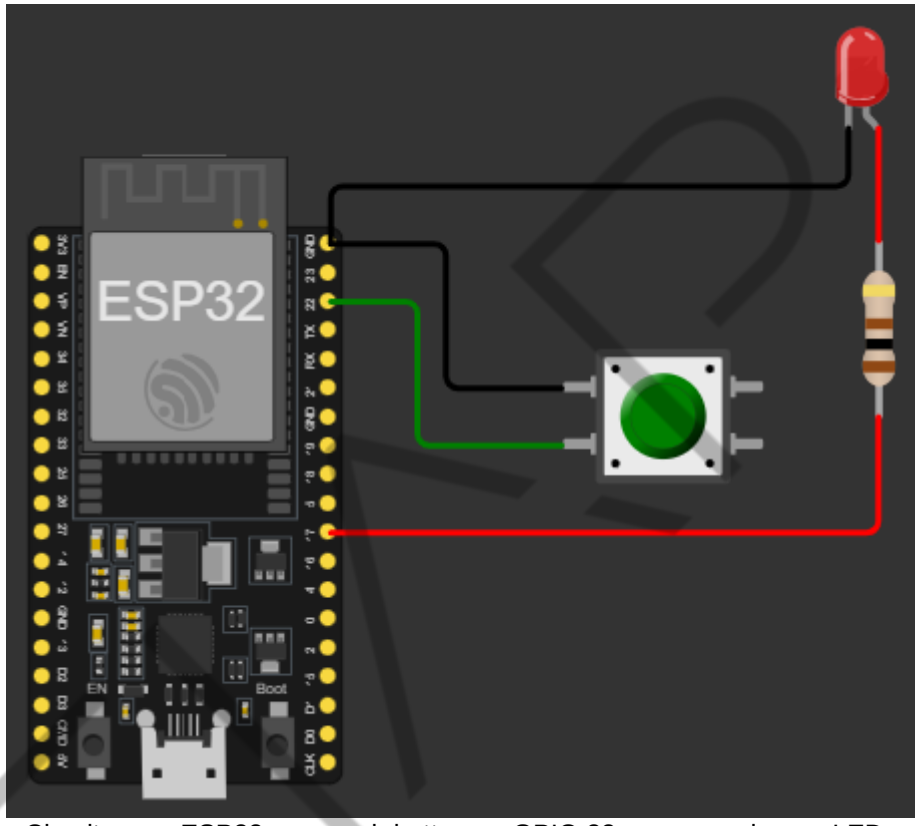

Figura 13 - Circuito com ESP32 com push button no GPIO 22 para acender um LED no GPIO 17
Fonte: Elaborado pelo autor (2024)

Quando for utilizar botões de pressão para controlar algo usando o ESP32, sempre siga essa regra: declare o pino GPIO que você escolheu para conectar o botão como:

`pinMode(pino, INPUT_PULLUP);`

Isso vai fazer com que internamente ao ESP32, o pino escolhido fique sempre ligado no nível HIGH, ou seja, no positivo. Então, esse pino estará sempre detectando True. E quando você pressionar o botão, note que ele vai chavear o pino escolhido para o GND (que é LOW ou False). Logo, você nunca terá ruídos entrando no pino do botão, pois ele sempre terá uma definição clara de True ou False.

Portanto, você precisa testar o False quando quiser saber se o botão foi pressionado pelo usuário. Lembre-se que um push button é um simples interruptor

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

que permite ou interrompe o fluxo de corrente em um circuito quando pressionado. Ele tem normalmente quatro terminais, mas em muitos casos, apenas dois são utilizados. Quando o botão é pressionado, os dois terminais são conectados internamente, permitindo que a corrente flua.

3.5.1 Pinagem

No esquema de conexão, um terminal do push button é conectado ao pino de entrada digital do ESP32 e o outro terminal é conectado ao GND, e você precisa ativar internamente um resistor de pull-up para manter o estado do pino em HIGH quando o botão não é pressionado.

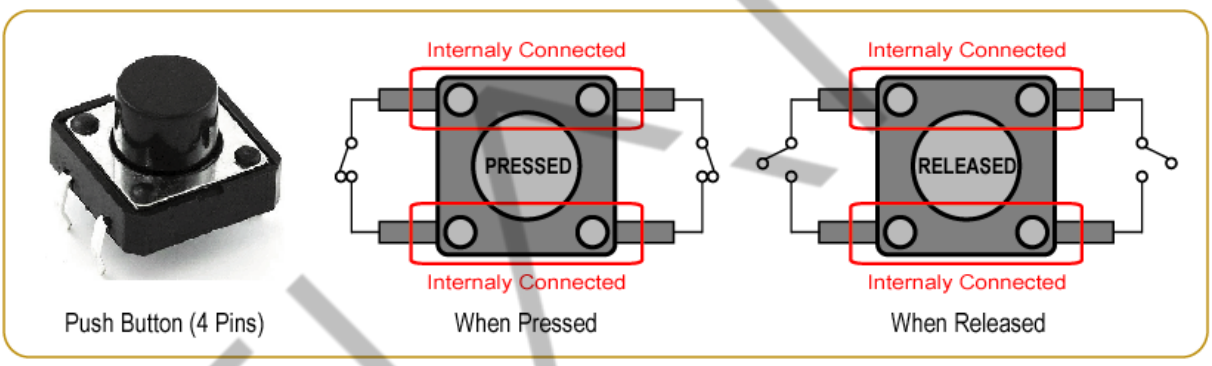

Figura 14 - Pinagem do push button quando é pressionado e quando não é pressionado
Fonte: Gmostofabd (2024)

3.5.2 Programação usando Push Button

A seguir está um código exemplo que detecta o estado de um push button e aciona uma ação correspondente que é ligar um LED:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
     #define buttonPin 22  // Pino GPIO conectado ao push button
     #define ledPin 17       // Pino GPIO conectado ao LED (ou
outro dispositivo)

     void setup() {
         pinMode(buttonPin, INPUT_PULLUP);  // Configura o pino
do botão como entrada com pull-up interno
         pinMode(ledPin, OUTPUT);           // Configura o pino
do LED como saída
         Serial.begin(9600);             // Inicia a comunicação
serial
     }

     void loop() {
         int buttonState = digitalRead(buttonPin);   // Lê o
estado do botão

         if (buttonState == LOW) {  // O botão está pressionado
(pino conectado ao GND)
             digitalWrite(ledPin, HIGH);  // Liga o LED
             Serial.println("Botão pressionado!");
         } else {   // O botão não está pressionado (pino em
estado HIGH)
             digitalWrite(ledPin, LOW);   // Desliga o LED
         }

         delay(50);   // Pequeno atraso para evitar ruído de
contato (debounce)
     }
```

Código-fonte 5 - Exemplo de como programar o botão de pressão
Fonte: Elaborado pelo autor (2024)

Explicação detalhada do que é novidade no código em relação aos exemplos anteriores. Para saber dos detalhes das linhas não comentadas aqui, retorne aos códigos anteriores.

- **`pinMode(buttonPin, INPUT_PULLUP);`** configura o pino do push button como entrada e ativa o resistor de pull-up interno. Isso garante que o pino estará em estado HIGH quando o botão não estiver pressionado.
- **`int buttonState = digitalRead(buttonPin);`** lê o estado atual do pino conectado ao botão. Se o botão estiver pressionado, o estado será LOW; caso contrário, será HIGH.
- **`if (buttonState == LOW)`** verifica se o botão está pressionado. Se sim, o LED é ligado e mantém-se aceso enquanto o botão fica pressionado e, ao

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

mesmo tempo, uma mensagem é enviada ao Monitor Serial. Quando o botão é liberado, o LED apaga e a mensagem para de aparecer no Monitor Serial.

- **`delay(50);`** adiciona um pequeno atraso para evitar múltiplas detecções devido ao ruído de contato (*debounce*) comum em botões mecânicos.

3.5.3 Aplicações do push button em sistemas de IA

Integrar um push button em um sistema controlado por ESP32 permite ao usuário interagir diretamente com modelos de inteligência artificial, facilitando a ativação ou desativação de funções inteligentes. Por exemplo:

- Em um sistema de IA, o push button pode ser utilizado para marcar eventos específicos, como capturar uma imagem, gravar um áudio, ou registrar dados de sensores quando o usuário pressiona o botão. Esses dados podem ser usados para treinar modelos de IA, como redes neurais, para reconhecer padrões ou realizar previsões.
- Em robótica ou automação residencial, o push button pode ser usado para alternar entre diferentes modos de operação de um sistema controlado por IA. Isto é, pressionar o botão uma vez pode ativar o modo de aprendizado, onde o robô coleta dados do ambiente, e pressionar novamente pode iniciar o modo autônomo, onde o robô toma decisões com base no modelo treinado.
- Em sistemas de IA, o push button pode servir como um interruptor de emergência para interromper o funcionamento do sistema em caso de falha ou comportamento inesperado. Isso é especialmente útil em ambientes industriais ou em robôs autônomos que podem precisar ser parados instantaneamente.
- O push button pode ser configurado para iniciar análises em tempo real em sistemas de IA. Isto é, em um sistema de reconhecimento facial, pressionar o botão pode acionar o ESP32 para capturar uma imagem e enviá-la ao modelo de IA para identificar uma pessoa.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

O uso de push buttons em projetos com ESP32 é uma maneira simples, mas poderosa, de integrar interatividade em sistemas de IA. Ao permitir que o usuário ou o sistema externo envie sinais para o microcontrolador, o push button pode desencadear uma série de ações, desde o controle de dispositivos simples até a ativação de processos complexos de IA.

Este conhecimento serve como base para desenvolver interfaces homem-máquina (HMI) melhores, em que os controles manuais e automáticos são integrados e sincronizados.

3.6 Sensor de movimento PIR

O sensor PIR (Passive Infrared) é um dispositivo que detecta movimento por meio da captação de variações na radiação infravermelha emitida por objetos no ambiente, como pessoas e animais.

Ele é amplamente utilizado em sistemas de segurança, automação residencial e dispositivos IoT. Quando combinado com um microcontrolador, como o ESP32, o sensor PIR é importante para criar sistemas que respondem ao movimento de forma inteligente.

Em projetos de IA, o sensor PIR pode ser utilizado para ativar modelos de inteligência artificial, coletar dados de presença ou iniciar processos de reconhecimento de padrões.

3.6.1 Funcionamento e pinagem do sensor PIR

O sensor PIR detecta movimento ao medir a mudança na radiação infravermelha (calor) no ambiente. Quando um objeto com temperatura diferente do ambiente (como um corpo humano) se move dentro do campo de visão do sensor, ele detecta essa mudança e gera um sinal de saída. Na figura “Sensor PIR de vários ângulos” mostra como é esse sensor.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Figura 15 - Sensor PIR de vários ângulos
Fonte: Proyecto Electronico (2017)

Observação: o pino S é para Saída de dado, G para Ground e D para Dreno ou VCC.

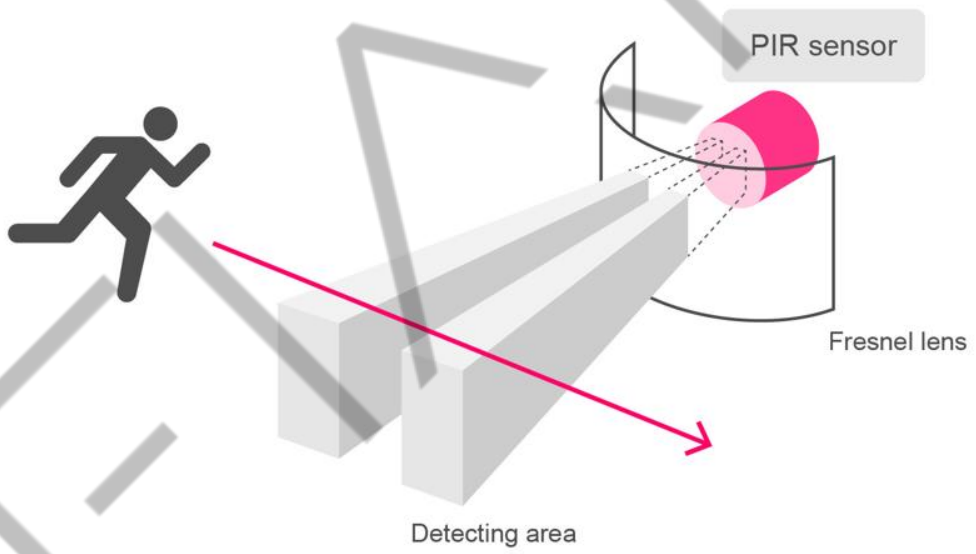

Figura 16 - Detecção do sensor usando uma lente que amplia o campo de visão e o sinal infravermelho detectado
Fonte: Mokolora (2022), adaptado por FIAP (2024)

Entretanto, o sensor que compramos facilmente no mercado, já possui uma eletrônica que facilita o seu uso, pois o sensor mostrado na figura “Sensor PIR de vários ângulos”, não está completo. É preciso de uma eletrônica que comunique com o sensor e libere um sinal quando algo é identificado. A figura “Sensor PIR com a eletrônica disponível no mercado” mostra como é o sensor final comercial.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

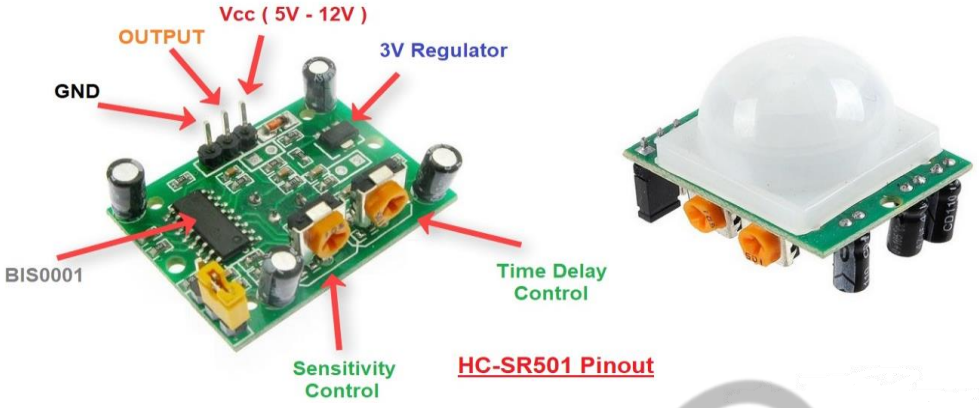

Figura 17 - Sensor PIR com a eletrônica disponível no mercado
Fonte: LinkedIn (2024)

Observe os pinos de alimentação e saída de dados. Veja também a lente de Fresnel que é utilizada para aumentar o campo de visão.

O sensor PIR tem um pino de saída que muda de estado (de LOW para HIGH) quando o movimento é detectado. Este sinal pode ser lido por um pino digital do ESP32 para acionar uma resposta programada.

Esquema de conexão:

- **VCC:** conectado a 5V ou 3.3V do ESP32.
- **GND:** conectado ao GND do ESP32.
- **OUT:** conectado a um pino GPIO do ESP32 (por exemplo, GPIO 17).

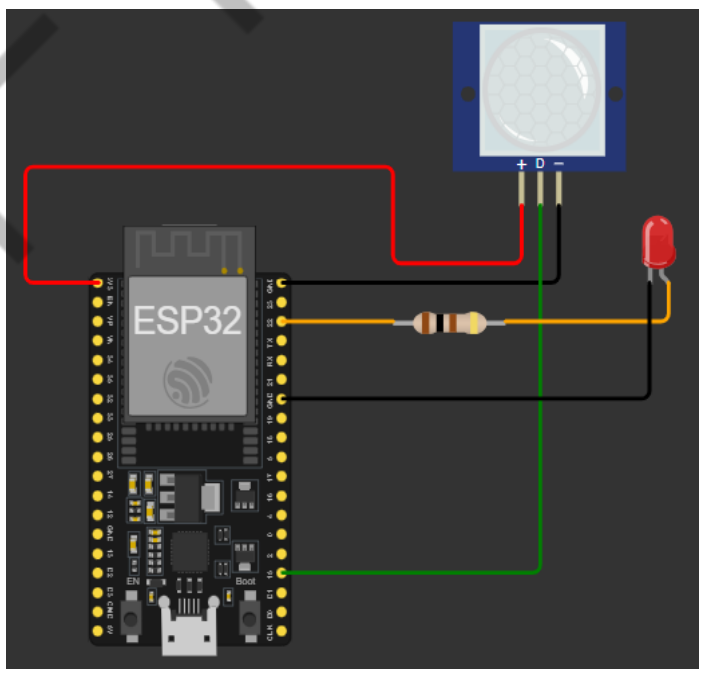

Figura 18 - Sensor PIR conectado no ESP32. LED para ser controlado
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

3.6.2 Programa usando PIR

A seguir é apresentado um código de exemplo, que detecta movimento usando um sensor PIR e exibe uma mensagem no Monitor Serial, além de acionar um LED:

```
     #define pinPIR 17        // Pino GPIO conectado ao pino OUT
do sensor PIR
     #define pinLED 22        // Pino GPIO conectado ao LED

     void setup() {
         pinMode(pinPIR, INPUT);        //  Configura  o  pino  do
sensor PIR como entrada
         pinMode(pinLED, OUTPUT);       //  Configura  o  pino  do
LED como saída
         Serial.begin(9600);                //     Inicia     a
comunicação serial
     }

     void loop() {
         int pirState = digitalRead(pinPIR);  // Lê o estado do
sensor PIR

         if (pirState == HIGH) {  // Movimento detectado
             digitalWrite(pinLED, HIGH);  // Liga o LED
             Serial.println("Movimento detectado!");
         } else {  // Nenhum movimento detectado
             digitalWrite(pinLED, LOW);   // Desliga o LED
         }

         delay(100);  // Pequeno atraso para suavizar a leitura
     }
```

Código-fonte 6 - Exemplo de como programar o uso do sensor PIR
Fonte: Elaborado pelo autor (2024)

Explicação do código:

- **`#define pinPIR 13`** define o pino GPIO 17 do ESP32 como `pinPIR`, que será conectado ao pino de saída (OUT) do sensor PIR. Este pino será usado para ler o estado do sensor, ou seja, se o movimento foi detectado ou não.
- **`#define pinLED 22`** define o pino GPIO 22 do ESP32 como `pinLED`, que será conectado ao LED. Este pino será utilizado para controlar o estado do LED, ou seja, ligar ou desligar com base na leitura do sensor PIR.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- **`void setup()`** a função **`setup()`** é executada uma vez, logo após o ESP32 ser ligado ou reiniciado. Nesta função, você configura os pinos e inicializa a comunicação serial.
- **`pinMode(pinPIR, INPUT);`** configura o pino `pinPIR` (GPIO 13) como uma entrada. Isso significa que o ESP32 vai ler o estado desse pino para determinar se o sensor PIR detectou movimento.
- **`pinMode(pinLED, OUTPUT);`** configura o pino `pinLED` (GPIO 22) como uma saída. Este pino será utilizado para ligar ou desligar o LED.
- **`Serial.begin(9600);`** inicia a comunicação serial a uma velocidade de 9600 bps (bits por segundo). Isso permite que o ESP32 envie dados para o Monitor Serial no IDE do Arduino, onde você poderá visualizar as mensagens de detecção de movimento.
- **`void loop()`** esta função é chamada repetidamente enquanto o ESP32 estiver ligado. Ela contém o código principal que será executado continuamente para detectar movimento e controlar o LED.
- **`int pirState = digitalRead(pinPIR);`** lê o estado do pino `pinPIR` (conectado ao sensor PIR). O valor retornado será `HIGH` (1) se o sensor detectar movimento, ou `LOW` (0) se não houver movimento.
- **`if (pirState == HIGH)`** verifica se o estado lido do sensor PIR é `HIGH`, indicando que o movimento foi detectado.
- **`digitalWrite(pinLED, HIGH);`** se o movimento for detectado, o LED é ligado, configurando o pino `pinLED` para `HIGH`.
- **`Serial.println("Movimento detectado!");`** envia uma mensagem para o Monitor Serial indicando que o movimento foi detectado.
- **`else { digitalWrite(pinLED, LOW); }`** se o sensor PIR não detectar movimento (estado `LOW`), o LED é desligado, configurando o pino `pinLED` para `LOW`.
- **`delay(100);`** introduz um pequeno atraso de 100 milissegundos entre as leituras. Isso ajuda a suavizar as leituras e evita que o sistema responda muito rapidamente a pequenas flutuações ou ruídos do sensor.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

3.6.3 Aplicações do sensor PIR em sistemas de IA

Integrar um sensor PIR em sistemas controlados pelo ESP32 oferece inúmeras possibilidades em projetos de IA. O sensor pode ser utilizado para detecção de presença, que por sua vez, pode desencadear uma série de ações inteligentes ou coleta de dados para análises posteriores. Exemplos:

- Em um ambiente inteligente, o sensor PIR pode ser utilizado para monitorar a atividade de pessoas em uma sala. Os dados de presença podem ser enviados para um modelo de IA que analisa padrões de movimento ao longo do tempo, permitindo a detecção de comportamentos anômalos ou padrões de ocupação.
- O sensor PIR pode ser integrado em sistemas de iluminação inteligente, onde a detecção de movimento aciona as luzes automaticamente. A IA pode aprender os hábitos dos moradores e ajustar o sistema de iluminação para economizar energia ou aumentar o conforto.
- Em sistemas de segurança, o sensor PIR pode acionar alarmes ou enviar notificações ao detectar movimento em áreas restritas. Um sistema de IA pode analisar os dados do PIR junto com outras fontes, como câmeras, para identificar intrusões e responder adequadamente.
- Em robótica, o sensor PIR pode ser utilizado para detectar obstáculos móveis ou humanos em uma área de trabalho. A IA pode usar essa informação para ajustar a rota do robô, evitando colisões e garantindo uma operação segura.

3.7 Buzzer e pinagem

O buzzer (campainha) é um componente eletrônico que emite som, usado em diversas aplicações para fornecer feedback auditivo ou alertar sobre eventos específicos.

Em projetos com o ESP32, o buzzer pode ser programado para emitir diferentes sons ou padrões de bipes, dependendo das condições definidas no código.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Quando combinado com IA, o buzzer pode atuar como um meio de comunicação com o usuário, sinalizando resultados de processamento, alertando sobre condições críticas ou até mesmo interagindo com outros dispositivos inteligentes.

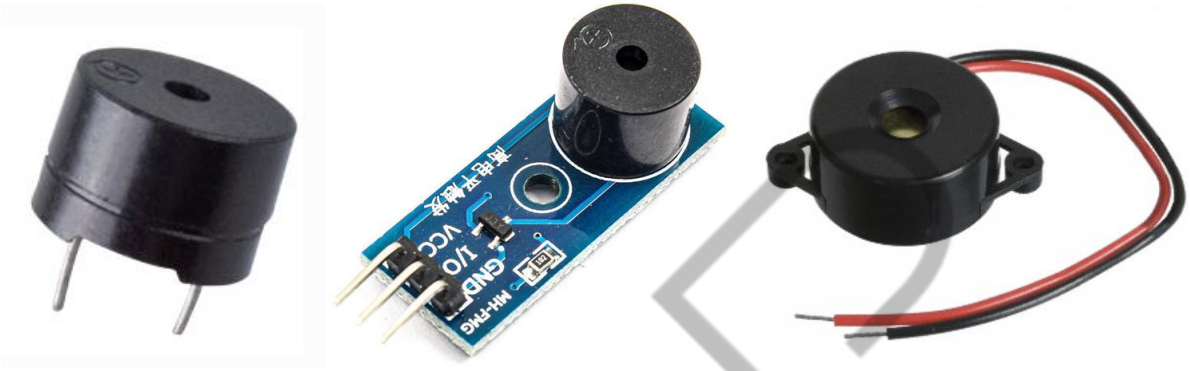

Figura 19 - Diferentes modelos de buzzer
Fonte: SunFounder (2022); Electro Peak! (2020); Autodesk Instructables (2015)

Na figura “Diferentes modelos de buzzer”, à esquerda temos o buzzer “nu”; no centro, ele já está numa plaquinha; e à direita é um buzzer com suporte e proteção.

Existem dois tipos principais de buzzers:

1. **Buzzer passivo:** requer um sinal de oscilação para gerar som. O ESP32 precisa fornecer um sinal PWM (Pulse Width Modulation) para controlar a frequência do som emitido.
2. **Buzzer ativo:** contém um oscilador interno e gera um tom constante quando alimentado. O controle do som é mais simples, pois basta fornecer uma tensão constante ao buzzer para que ele emita o som.

Esquema de conexão:

- **VCC:** conectado ao pino de alimentação (geralmente 5V ou 3.3V).
- **GND:** conectado ao GND do ESP32.
- **Pino de controle:** conectado a um pino GPIO do ESP32 (por exemplo, GPIO 15). O pino 15 suporta tanto o buzzer passivo quanto o ativo. Caso o pino 15 do seu projeto esteja ocupado com outro sensor, pode escolher outro que também vai funcionar. Praticamente, todos os pinos do ESP32 suportam o sinal PWM.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

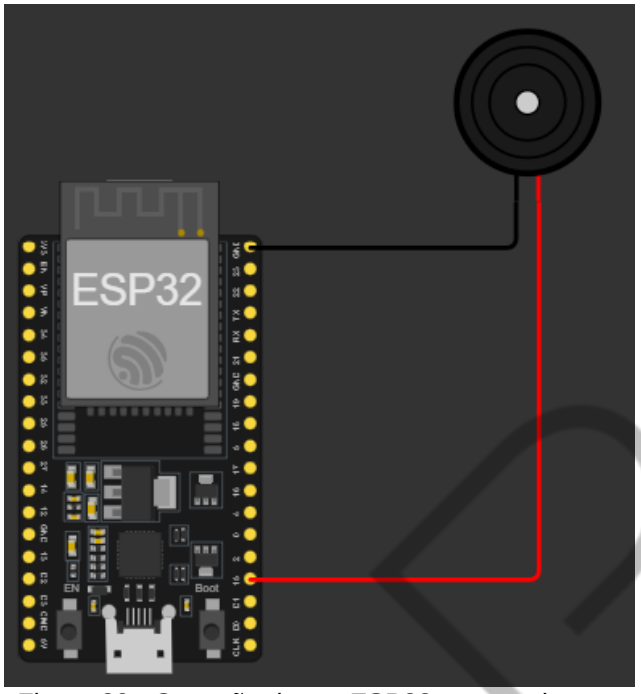

Figura 20 - Conexão de um ESP32 com um buzzer
Fonte: Elaborado pelo autor (2024)

3.7.1 Programando com o buzzer

Aqui está um exemplo de código que usa um buzzer passivo para emitir sons de diferentes frequências, criando uma melodia simples:

```
#define buzzerPin 15  // Pino GPIO conectado ao buzzer

void setup() {
    pinMode(buzzerPin, OUTPUT);   // Configura o pino do buzzer como saída
}

void loop() {
    tone(buzzerPin, 1000);    // Emite um som de 1000 Hz (1 kHz)
    delay(500);                    // Aguarda meio segundo
    noTone(buzzerPin);        // Para o som
    delay(500);                    // Pausa antes do próximo som

    tone(buzzerPin, 1500);    // Emite um som de 1500 Hz
    delay(500);
    noTone(buzzerPin);
    delay(500);
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
        tone(buzzerPin, 2000);    // Emite um som de 2000 Hz
        delay(500);
        noTone(buzzerPin);
        delay(500);
    }
```

Código-fonte 7 - Exemplo de buzzer emitindo sons em frequências diferentes
Fonte: Elaborado pelo autor (2024)

Explicação do código:

- **`pinMode(buzzerPin, OUTPUT);`** configura o pino conectado ao buzzer como uma saída, permitindo que o ESP32 controle o buzzer.
- **`tone(buzzerPin, 1000);`** usa a função `tone()` para gerar um som de 1000 Hz no buzzer. Esta função é útil para buzzers passivos, onde você precisa controlar a frequência.
- **`delay(500);`** aguarda 500 milissegundos antes de continuar o código. Aqui, é usado para controlar a duração do som.
- **`noTone(buzzerPin);`** para o som que está sendo gerado pelo buzzer. Sem esta linha, o buzzer continuaria a emitir som indefinidamente.

3.7.2 Aplicações do buzzer em sistemas de IA

O buzzer pode ser utilizado de várias maneiras em sistemas de IA, proporcionando uma interface auditiva para feedback e alertas. A seguir estão algumas aplicações típicas:

- Em sistemas de IA que executam tarefas demoradas, como treinamento de modelos ou análise de grandes volumes de dados, o buzzer pode ser usado para notificar o usuário quando o processamento é concluído. O ESP32 pode monitorar o progresso e acionar o buzzer ao final da tarefa.
- Em sistemas de monitoramento que utilizam IA para detectar anomalias, o buzzer pode ser acionado quando um evento fora do comum é detectado, como a presença de um intruso ou a detecção de falhas em equipamentos. Esse feedback imediato pode ser crucial para a segurança e a manutenção.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- Em robótica ou sistemas automáticos, o buzzer pode ser usado para indicar o estado operacional de um dispositivo. Diferentes padrões de som podem indicar, por exemplo, que o sistema está inicializando, funcionando corretamente, ou encontrou um erro.
- Em aplicações de IA interativas, como reconhecimento de voz ou comandos de controle por gestos, o buzzer pode ser usado para confirmar que um comando foi recebido e processado. Por exemplo, um som curto pode indicar que o sistema reconheceu uma palavra ou gesto.

3.8 Servomotor

Os servomotores são componentes utilizados em robótica, automação e sistemas de controle, conhecidos por sua precisão no controle de movimento.

Eles são amplamente utilizados em projetos onde é necessário controlar o ângulo de rotação de um eixo, como em braços robóticos, mecanismos de direção de veículos e sistemas de controle de posição.

Com o ESP32, o servomotor pode ser programado para se mover a ângulos específicos, responder a dados de sensores, ou até mesmo integrar-se a sistemas de IA para executar tarefas complexas de maneira automatizada.

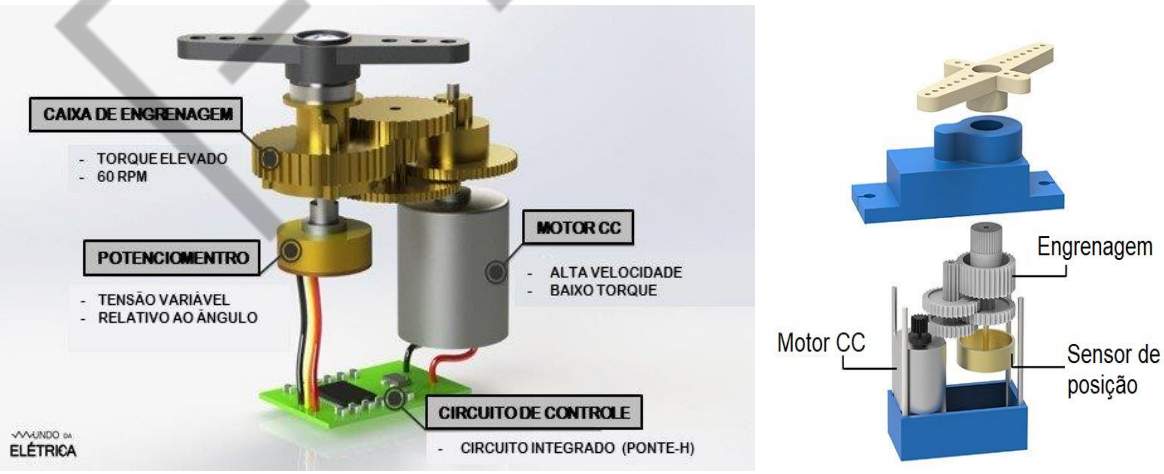

Figura 21 - Desenhos que explicam o servomotor internamente
Fonte: LinkedIn (2022); Mundo da elétrica (2024)

A figura a seguir mostra o servomotor real, modelo Sg90 da Tower Pro:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

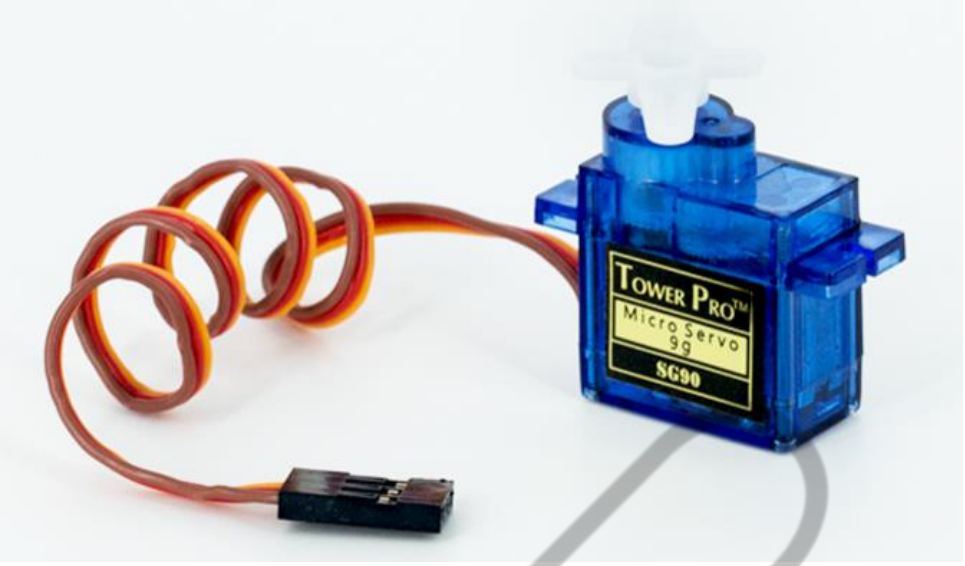

Figura 22 - Servomotor Sg90 Tower Pro
Fonte: Fazedores (2018)

Como podemos observar na figura, ele possui três pinos: VCC (vermelho), GND (marrom) e PWM (laranja).

Na figura a seguir, tem-se a conexão do servomotor com o ESP32 no ambiente do Wokwi:

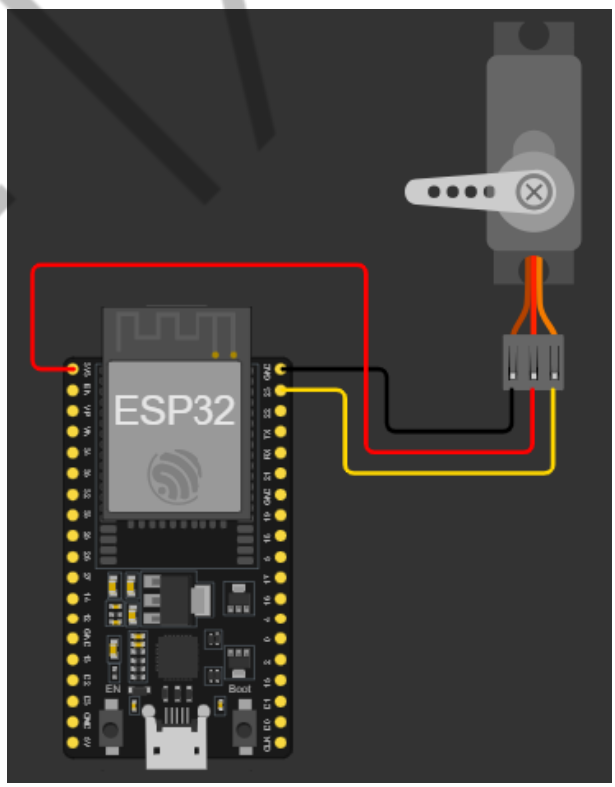

Figura 23 - Conexão do servomotor com o ESP32
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

3.8.1 Funcionamento de um servomotor

Um servomotor é um atuador rotativo que permite o controle preciso de ângulo, velocidade e posição. Ele é composto por um motor DC, um conjunto de engrenagens de redução, um potenciômetro para feedback de posição e um circuito de controle.

Os servomotores são controlados através de um sinal PWM. O que é um sinal PWM?

O sinal PWM é uma técnica utilizada para controlar a potência entregue a um dispositivo eletrônico, como motores, LEDs, ou servomotores, ajustando a largura de pulsos em um sinal digital. Em um sinal PWM, a tensão é alternada rapidamente entre dois níveis: alto (HIGH, geralmente 5V ou 3.3V) e baixo (LOW, 0V).

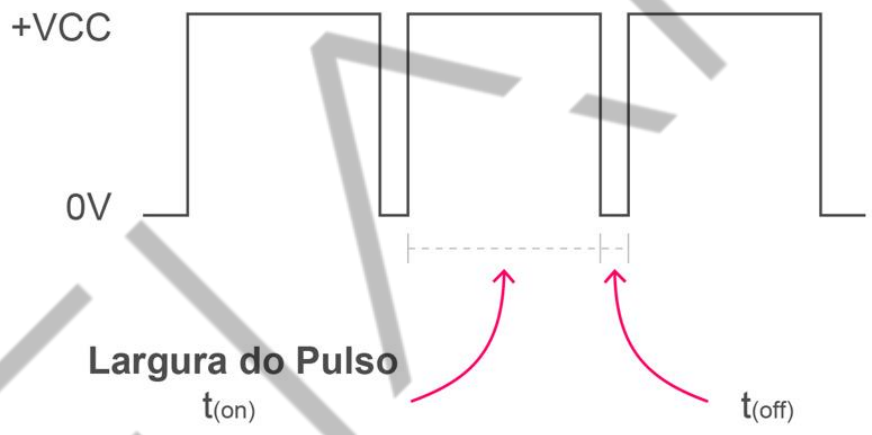

Figura 24 - Sinal PWM com duty próximo dos 90%, isto é, quase toda a onda está em nível On
Fonte: Manual da Eletrônica (2024), adaptado por FIAP (2024)

O ângulo de rotação do servomotor é determinado pela largura do pulso enviado ao pino de controle:

- **Pulso de 1 ms para t_{on} :** move o servo para a posição mínima (0 graus).
- **Pulso de 1,5 ms para t_{on}:** move o servo para a posição intermediária (90 graus).
- **Pulso de 2 ms para t_{on}:** move o servo para a posição máxima (180 graus). O servomotor só consegue chegar até 180º.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

O sinal PWM possui um parâmetro chamado Período que é o tempo total de um ciclo completo de sinal, composto por um tempo em nível alto (HIGH) e um tempo em nível baixo (LOW). Já o Duty Cycle (ciclo de trabalho) é a proporção do tempo em que o sinal está em nível alto (HIGH) durante um ciclo. Vale mencionar que o Duty Cycle é expresso em porcentagem. Por exemplo:

- Um Duty Cycle de 50% significa que o sinal está em nível alto metade do tempo e em nível baixo na outra metade do ciclo.
- Um Duty Cycle de 25% significa que o sinal está em nível alto durante um quarto do ciclo e em nível baixo durante os três quartos restantes.

O PWM é usado para controlar a quantidade de energia fornecida a um dispositivo. Embora o sinal PWM seja digital (apenas dois estados: alto e baixo), ele pode simular uma saída analógica variando o tempo em que o sinal permanece em nível alto. Isso é útil para controlar dispositivos que exigem ajustes finos de potência ou velocidade, como motores, LEDs de brilho variável, ou o posicionamento de servomotores.

Exemplo com LEDs:

- Duty Cycle de 100% significa que o LED estará totalmente aceso, pois o sinal está sempre em nível alto.
- Duty Cycle de 50% indica que o LED estará a metade do brilho máximo, pois o sinal está alto apenas metade do tempo.
- Duty Cycle de 0% mostra que o LED estará completamente apagado, pois o sinal nunca estará em nível alto.

A frequência do PWM refere-se à rapidez com que os ciclos completos de HIGH e LOW ocorrem. Frequências mais altas resultam em ciclos mais curtos. A escolha da frequência depende da aplicação, mas para servomotores, tipicamente operam com uma frequência de PWM de 50 Hz, porém, o que importa é a largura do pulso e não a frequência.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

3.8.2 Programando com servomotor

Segue um exemplo de código que utiliza a biblioteca Servo para controlar o ângulo de um servomotor usando o ESP32:

```
     #include <Servo.h>

     Servo myServo;  // Cria um objeto Servo

     #define servoPin 23  // Pino GPIO conectado ao sinal do
servomotor

     void setup() {
         myServo.attach(servoPin);  // Conecta o servo ao pino
especificado
     }

     void loop() {
         for (int angle = 0; angle <= 180; angle += 1) {  //
Gira o servo de 0 a 180 graus
             myServo.write(angle);  // Define o ângulo do servo
             delay(15);  // Espera 15 ms para o servo alcançar
a posição
         }
         delay(1000);  // Pausa de 1 segundo

         for (int angle = 180; angle >= 0; angle -= 1) {  //
Gira o servo de 180 a 0 graus
             myServo.write(angle);  // Define o ângulo do servo
             delay(15);  // Espera 15 ms para o servo alcançar
a posição
         }
         delay(1000);  // Pausa de 1 segundo
     }
```

Código-fonte 8 - Exemplo de código para controlar servomotor
Fonte: Elaborado pelo autor (2024)

Agora veremos uma explicação detalhada do código:

- **`#include <Servo.h>`** esta linha inclui a biblioteca "Servo.h", que contém as funções necessárias para controlar servomotores. Esta biblioteca simplifica o controle de servos ao lidar automaticamente com a geração dos sinais PWM necessários para controlar o ângulo do servo.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- **`Servo myServo;`** gera um objeto do tipo `Servo` chamado `myServo`. Este objeto será utilizado para controlar o servomotor, permitindo que você ajuste o ângulo do motor por meio das funções da biblioteca.
- **`#define servoPin 23`** define uma constante chamada `servoPin`, que representa o pino GPIO 23 do ESP32. Este pino é onde o fio de sinal do servomotor será conectado. O ESP32 enviará sinais PWM por meio deste pino para controlar o ângulo do servo.
- **`void setup()`** a função **`setup()`** é executada uma vez, logo após o ESP32 ser ligado ou reiniciado. Ela é usada para inicializar configurações e preparar o sistema para execução.
- **`myServo.attach(servoPin);`** esta linha associa o objeto `myServo` ao pino especificado (GPIO 23). A partir deste ponto, qualquer comando enviado ao objeto `myServo` controlará o servomotor conectado a este pino.
- **`void loop()`** esta função é chamada repetidamente enquanto o ESP32 estiver ligado. Ela contém o código principal que será executado continuamente.
- **`for (int angle = 0; angle <= 180; angle += 1)`** um loop que começa com o ângulo igual a 0 e o incrementa em 1 grau até que alcance 180 graus. Este loop move o servomotor do ângulo de 0 graus até 180 graus gradualmente.
- **`myServo.write(angle);`** define o ângulo do servomotor para o valor atual de `angle`. O servo se moverá para o ângulo especificado, conforme o loop incrementa o valor de `angle`.
- **`delay(15);`** espera 15 milissegundos para permitir que o servo se mova para a nova posição antes de incrementar o ângulo novamente. Este pequeno atraso é necessário porque os servomotores levam tempo para alcançar a nova posição.
- **`delay(1000);`** introduz uma pausa de 1 segundo após o servo atingir 180 graus. Isso faz com que o servo permaneça na posição final por um tempo antes de começar a voltar para 0 graus.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- **`for (int angle = 180; angle >= 0; angle -= 1)`** um loop que começa com o ângulo igual a 180 e o decrementa em 1 grau até que alcance 0 graus. Este loop move o servomotor de volta de 180 graus para 0 graus.
- **`myServo.write(angle);`** define o ângulo do servomotor para o valor atual de `angle`, movendo o servo de volta para 0 graus.
- **`delay(15);`** introduz uma pequena pausa de 15 milissegundos para garantir que o servo tenha tempo para se mover para a nova posição antes de mudar novamente o ângulo.
- **`delay(1000);`** introduz outra pausa de 1 segundo após o servo atingir 0 graus. Isso faz com que o servo permaneça na posição inicial por um tempo antes de começar a mover-se novamente.

3.8.3 Aplicações do servomotor em sistemas de IA

Os servomotores são utilizados em aplicações que requerem controle preciso de movimento. Quando integrados a sistemas de IA, eles permitem que máquinas e dispositivos funcionem de forma inteligente e autônoma, respondendo dinamicamente a dados de sensores e a comandos complexos.

- Em robôs que utilizam IA para realizar tarefas como montagem de peças ou manipulação de objetos, os servomotores são responsáveis por mover os braços com precisão. A IA pode determinar a posição necessária com base em algoritmos de visão computacional ou aprendizado de máquina, enquanto os servomotores executam o movimento.
- Em veículos autônomos, servomotores são usados para controlar a direção das rodas. A IA processa dados de sensores, como LIDAR e câmeras, para tomar decisões de direção, e o servomotor ajusta a direção do veículo de acordo com essas decisões.
- Servomotores podem ser utilizados em sistemas de automação residencial para controlar cortinas ou janelas. A IA pode aprender os hábitos dos moradores e ajustar as cortinas automaticamente com base na hora do dia ou nas condições climáticas.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- Em sistemas onde a IA interage diretamente com os usuários, servomotores podem ser úteis para criar interfaces tangíveis que respondem fisicamente a comandos ou mudanças de estado. Por exemplo, em assistentes robóticos, o movimento de braços ou cabeças pode ser controlado por servomotores, proporcionando uma interação mais natural e expressiva.

3.9 Sensor de luz LDR

O LDR (Light Dependent Resistor) é um componente eletrônico que altera sua resistência em resposta à intensidade da luz. Quando a luz incide sobre o LDR, sua resistência diminui, permitindo maior passagem de corrente; em ambientes escuros, sua resistência aumenta, reduzindo a corrente que passa por ele.

Essa propriedade torna o LDR ideal para medir a luminosidade em diferentes ambientes, sendo muito utilizado em projetos de automação residencial, dispositivos IoT e sistemas de monitoramento ambiental.

Quando combinado com o ESP32 e sistemas de IA, o LDR pode ser útil para criar soluções inteligentes que respondem dinamicamente às mudanças de iluminação.

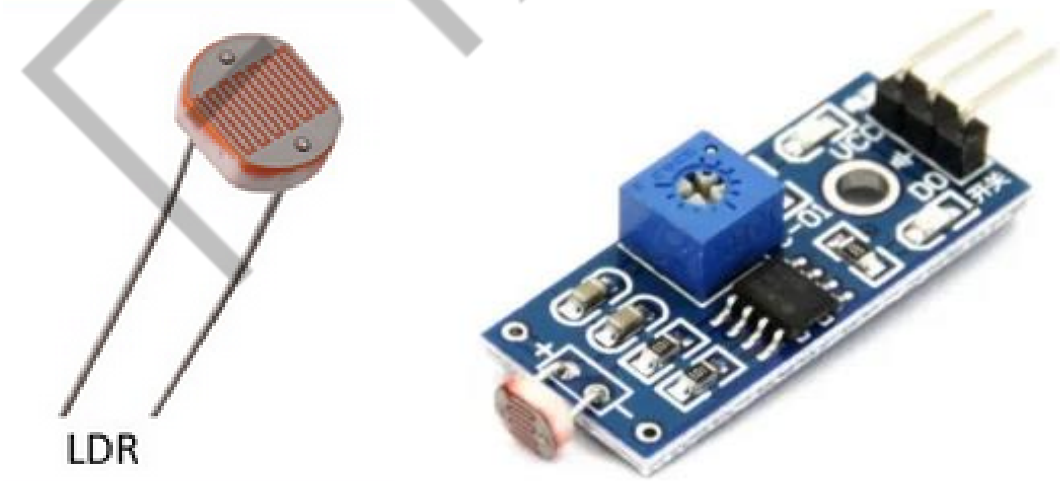

Figura 25 - Sensor LDR “nu” à esquerda; sensor LDR em forma de placa à direita
Fonte: STA (2020); DIY projects lab (2022)

Observação: os pinos VCC e ⏚ são para a alimentação, e o pino D0 é para a saída de dados.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

O LDR é composto por um material semicondutor cuja resistência varia inversamente com a intensidade da luz. Em termos simples:

- **Alta luz (alta intensidade):** baixa resistência, mais corrente passa através do LDR.
- **Baixa luz (baixa intensidade):** alta resistência, menos corrente passa através do LDR.

Essa característica permite que o LDR seja utilizado como um sensor de luz, gerando um valor analógico que pode ser lido por um microcontrolador como o ESP32.

O LDR é frequentemente usado em um divisor de tensão, onde sua resistência variável altera a tensão de saída do circuito em função da luz ambiente. Essa tensão pode ser medida pelo ESP32 por meio de um pino de entrada analógica (ADC).

3.9.1 Pinagem do LDR

Segue um exemplo de como se liga um módulo de LDR no ESP32.

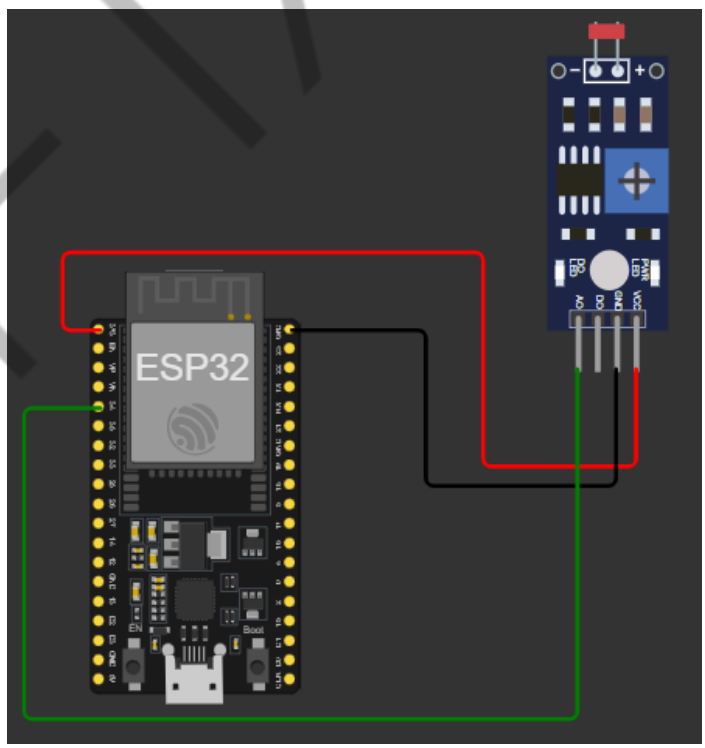

Figura 26 - Circuito ESP32 com sensor LDR em forma de módulo
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

3.9.2 Programando com o LDR

Esse é um exemplo de código para ler os valores de luminosidade de um LDR usando o ESP32. Observe que dessa vez, mudamos para o pino GPIO 34. Lembre-se de sempre dar preferência para os pinos da esquerda do ESP32 quando quiser analisar sinais analógicos, e quando quiser analisar sinais digitais, dê preferência para os pinos da direita do ESP32.

```
     #define ldrPin 34        // Pino ADC conectado ao LDR

     void setup() {
         Serial.begin(9600);       //   Inicia   a   comunicação
serial
     }

     void loop() {
         int  ldrValue  =  analogRead(ldrPin);   //  Lê  o  valor
analógico do LDR

         Serial.print("Nível de Luz: ");
         Serial.println(ldrValue);   // Exibe o valor lido no
Monitor Serial

         delay(500);  // Pequeno atraso para a próxima leitura
     }
```

Código-fonte 9 - Exemplo de como programar o LDR
Fonte: Elaborado pelo autor (2024)

Explicação do código:

- **`analogRead(ldrPin);`** lê o valor analógico do pino `ldrPin`, que representa a tensão no divisor de tensão formado pelo LDR e o resistor. Este valor está relacionado à intensidade da luz que incide sobre o LDR.
- **`Serial.print("Nível de Luz: ");`** e **`Serial.println(ldrValue);`** exibem o valor lido no Monitor Serial, permitindo monitorar a variação da luz ambiente em tempo real.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

3.9.3 Aplicação com LDR no mundo da IA

O LDR pode ser útil em diversas aplicações inteligentes que requerem detecção de luz ou controle baseado em luminosidade. A seguir estão algumas possíveis aplicações em sistemas de IA:

- O LDR pode ser utilizado em sistemas de automação residencial para controlar as luzes com base na luminosidade do ambiente. Um sistema de IA pode aprender os padrões de uso da luz em uma casa e ajustar automaticamente a iluminação para economizar energia ou melhorar o conforto dos moradores.
- Em projetos de agricultura de precisão, o LDR pode ser usado para monitorar os níveis de luz solar que atingem as plantas, em seguida, a IA pode analisar esses dados para ajustar sistemas de irrigação ou sombra, otimizando o crescimento das plantas.
- Em sistemas de segurança, o LDR pode detectar mudanças abruptas na luz, como quando uma porta ou cortina é aberta, acionando alarmes e notificações. Assim, a IA pode determinar se essa mudança de luz indica uma atividade normal ou uma possível intrusão.
- Em robôs que navegam em ambientes com variação de luz, o LDR pode ser utilizado para seguir a luz ou evitar áreas escuras. Enquanto a IA pode tomar decisões em tempo real com base nos dados do LDR, ajustando a rota do robô para alcançar um objetivo específico.

3.10 Sensor acelerômetro MPU6040

O sensor MPU6050 é um módulo que combina um acelerômetro de três eixos e um giroscópio de três eixos em um único dispositivo. Isso permite a medição da aceleração e da rotação em três dimensões, tornando-o essencial em aplicações que requerem a detecção de movimento, orientação e posição.

O MPU6050 é utilizado em robótica, dispositivos vestíveis, drones e sistemas de navegação. Quando integrado ao ESP32 e a sistemas de IA, o MPU6050 permite a criação de sistemas inteligentes que respondem dinamicamente aos movimentos,

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

otimizando tarefas como controle de equilíbrio, navegação autônoma e reconhecimento de gestos.

Como mencionado, o MPU6050 mede a aceleração e a rotação nos eixos X, Y e Z, fornecendo dados que podem ser úteis para calcular a inclinação, a velocidade angular e a orientação do dispositivo.

O sensor se comunica com o ESP32 por meio do protocolo I2C, um protocolo de comunicação serial que permite a transferência eficiente de dados entre dispositivos.

A figura “Sensor MPU6050” mostra o MPU6050 no mundo real:

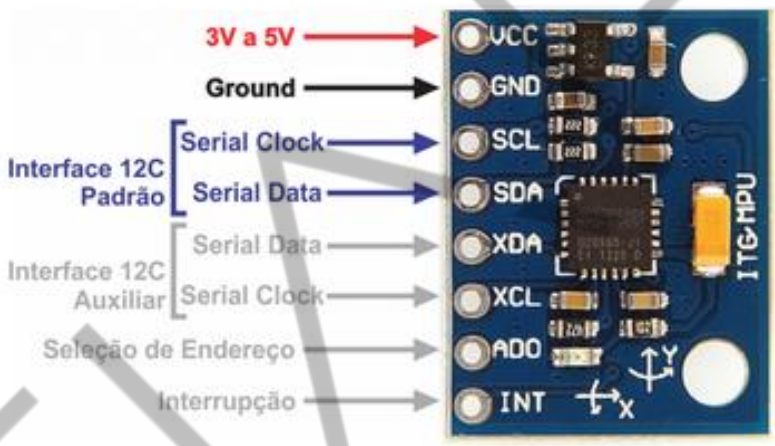

Figura 27 - Sensor MPU6050
Fonte: Arduino Ômega Blog (2021)

3.10.1 Acelerômetro

Mede a aceleração em três direções, nos eixos X, Y e Z, podendo detectar movimentos em qualquer direção no espaço tridimensional, e a unidade de medida da aceleração é geralmente medida em metros por segundo ao quadrado (m/s^2) ou em gravidade terrestre (g).

3.10.2 Giroscópio

Também mede nos eixos X, Y e Z, além de medir a rotação ao redor deles. Sendo útil para detectar mudanças de orientação e movimento angular.

A rotação é medida em graus por segundo (°/s).

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

3.10.3 Pinagem do MPU6050

Para conectar o MPU6050 ao ESP32, faça conforme ilustra a figura a “Conexão entre ESP32 e o MPU6050”.

- **VCC:** conecte ao pino 3.3V do ESP32.
- **GND:** conecte ao GND do ESP32.
- **SCL (Clock I2C):** conecte ao GPIO 22 do ESP32.
- **SDA (Dados I2C):** conecte ao GPIO 21 do ESP32.

Nota: o ESP32 usa níveis lógicos de 3.3V, compatíveis com o MPU6050.

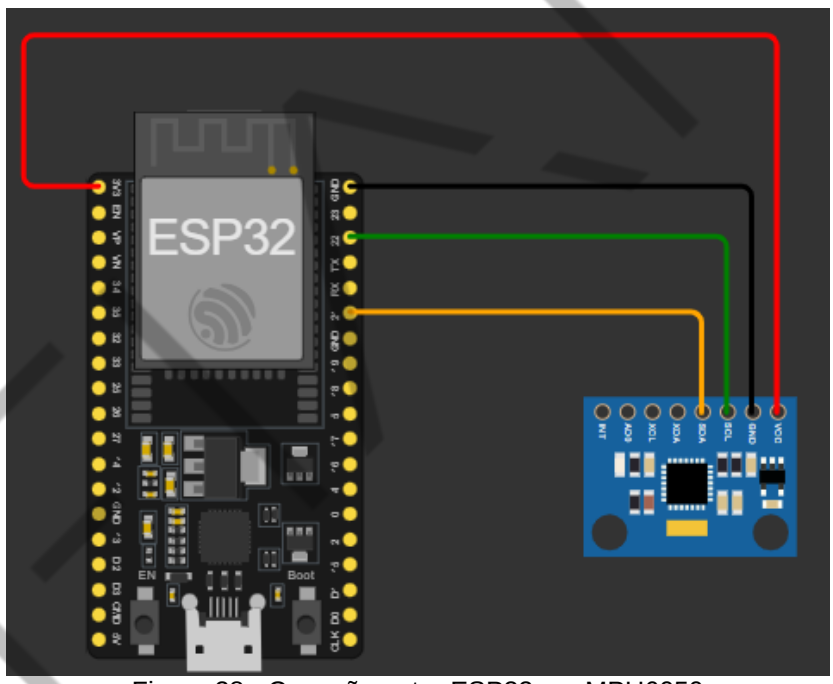

Figura 28 - Conexão entre ESP32 e o MPU6050
Fonte: Elaborado pelo autor (2024)

3.10.4 Programando com MPU6050

Aqui está um exemplo de código que utiliza a biblioteca “MPU6050” para ler os valores de aceleração e rotação do sensor:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
#include <Adafruit_MPU6050.h>
#include <Adafruit_Sensor.h>
#include <Wire.h>


Adafruit_MPU6050 mpu;

void setup(void) {
  Serial.begin(115200);

  if (!mpu.begin()) {
    Serial.println("Falha ao encontrar o chip MPU6050");
    while (1) {
      delay(10);
    }
  }

  mpu.setAccelerometerRange(MPU6050_RANGE_8_G);
  mpu.setGyroRange(MPU6050_RANGE_250_DEG);
  mpu.setFilterBandwidth(MPU6050_BAND_21_HZ);
  Serial.println("");
  delay(1000);
}

void loop() {

  /* Get new sensor events with the readings */
  sensors_event_t a, g, temp;
  mpu.getEvent(&a, &g, &temp);

  Serial.println("Acelerometro:");
  Serial.print(a.acceleration.x);
  Serial.print(",");
  Serial.print(a.acceleration.y);
  Serial.print(",");
  Serial.println(a.acceleration.z);
  Serial.println("");

  Serial.println("Giroscopio:");
  Serial.print(g.gyro.x);
  Serial.print(",");
  Serial.print(g.gyro.y);
  Serial.print(",");
  Serial.print(g.gyro.z);
  delay(10);
}
```

Código-fonte 10 - Exemplo de uso do sensor giroscópio
Fonte: Elaborado pelo autor (2024)

Explicação do código:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- **#include <Adafruit_MPU6050.h>** inclui a biblioteca "Adafruit_MPU6050", que contém funções para facilitar a comunicação e a leitura de dados do sensor MPU6050. Esta biblioteca abstrai muitos detalhes técnicos, tornando o uso do sensor mais simples e intuitivo.
- **#include <Adafruit_Sensor.h>** inclui a biblioteca "Adafruit_Sensor", que fornece uma interface comum para diferentes tipos de sensores. É utilizada junto com a biblioteca do MPU6050 para lidar com os dados do sensor de forma estruturada.
- **#include <Wire.h>** inclui a biblioteca "Wire", que é necessária para a comunicação I2C entre o ESP32 e o MPU6050. O I2C é um protocolo de comunicação que permite a conexão de múltiplos dispositivos em dois fios: um para o clock (SCL) e outro para os dados (SDA).
- **Adafruit_MPU6050 mpu;** cria um objeto mpu do tipo Adafruit_MPU6050, que será utilizado para interagir com o sensor MPU6050.
- **Serial.begin(115200);** inicializa a comunicação serial a uma taxa de 115200 bits por segundo (baud rate). Isso permite que o ESP32 envie dados ao Monitor Serial, onde você pode visualizar as leituras do sensor.
- **if (!mpu.begin()) { ... }** esta linha tenta inicializar o sensor MPU6050 usando o método begin(). Se a inicialização falhar (ou seja, se o sensor não for encontrado), o código dentro do bloco if é executado.
- **Serial.println("Falha ao encontrar o chip MPU6050");** exibe uma mensagem de erro no Monitor Serial indicando que o sensor MPU6050 não foi encontrado.
- **while (1) { delay(10); }** entra em um loop infinito, onde o código fica preso, com pequenos atrasos de 10 milissegundos entre cada ciclo. Isso interrompe o programa se o sensor não for detectado, evitando a execução de código sem o sensor presente.
- **mpu.setAccelerometerRange(MPU6050_RANGE_8_G);** configura o intervalo de medição do acelerômetro para ±8g (g é a unidade de gravidade, equivalente a aproximadamente 9,8 m/s²). Isso define a sensibilidade do acelerômetro.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- **mpu.setGyroRange(MPU6050_RANGE_250_DEG);** configura o intervalo de medição do giroscópio para ±250 graus por segundo. Isso define a sensibilidade do giroscópio.
- **mpu.setFilterBandwidth(MPU6050_BAND_21_HZ);** define a largura de banda do filtro do sensor para 21 Hz. Essa configuração ajuda a filtrar ruídos de alta frequência, tornando as leituras mais estáveis e precisas.
- **delay(1000);** introduz um atraso de 1 segundo para garantir que todas as configurações sejam aplicadas corretamente antes de iniciar a leitura dos dados.
- **sensors_event_t a, g, temp;** declara três variáveis do tipo sensors_event_t, que são usadas para armazenar os dados de aceleração (a), giroscópio (g) e temperatura (temp) capturados pelo sensor.
- **mpu.getEvent(&a, &g, &temp);** chama a função getEvent() para capturar os dados atuais do sensor e armazená-los nas variáveis a, g e temp. Essa função coleta os valores de aceleração, rotação e temperatura em um único passo.
- **Serial.println("Acelerometro:");** exibe o título "Acelerometro:" no Monitor Serial para indicar que os valores seguintes correspondem à aceleração medida pelo sensor.
- **Serial.print(a.acceleration.x);** e **Serial.print(",");** exibe os valores da aceleração nos eixos X, Y e Z, separados por vírgulas. Isso facilita a leitura e o registro dos dados.
- **Serial.println("Giroscopio:");** exibe o título "Giroscopio:" no Monitor Serial para indicar que os valores seguintes correspondem à rotação medida pelo giroscópio.
- **Serial.print(g.gyro.x);** e **Serial.print(",");** exibe os valores da rotação nos eixos X, Y e Z, também separados por vírgulas.
- **delay(10);** introduz um pequeno atraso de 10 milissegundos antes de repetir o loop, garantindo que as leituras sejam estáveis e evitando sobrecarregar a comunicação serial.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

3.10.5 Aplicações do MPU6050 em sistemas de IA

O MPU6050 pode ser utilizado em diversas aplicações que requerem a detecção de movimento e orientação. Quando integrado a sistemas de IA, ele permite que dispositivos tomem decisões inteligentes com base em dados de movimento e rotação. Exemplos:

- Em drones, o MPU6050 é crucial para manter a estabilidade. A IA pode processar os dados de aceleração e rotação para ajustar os motores em tempo real, garantindo um voo estável mesmo em condições adversas.
- O MPU6050 pode ser usado para capturar movimentos de mão ou corpo. Enquanto a IA pode reconhecer padrões nesses dados, permitindo o controle de dispositivos através de gestos, como mudar slides em uma apresentação ou controlar o volume de um dispositivo.
- Em robôs bípedes ou de duas rodas, o MPU6050 é útil para manter o equilíbrio. Enquanto a IA ajusta o posicionamento e o movimento do robô com base nos dados de inclinação e rotação fornecidos pelo sensor.
- Em dispositivos de realidade aumentada (AR) e realidade virtual (VR), o MPU6050 é utilizado para rastrear os movimentos da cabeça ou do controlador, permitindo uma experiência imersiva. Assim, a IA pode melhorar essa experiência prevendo e ajustando o conteúdo visual com base nos movimentos detectados.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

REFERÊNCIAS

ADMIN. How to Use an LDR Sensor with Arduino Tutorial. **DIY projects lab**, 24 ago. 2022. Disponível em: <https://diyprojectslab.com/ldr-sensor-with-arduino/>. Acesso em: 10 set. 2024.

ALVES, Pedro. O que é PWM? Funcionamento e aplicações. **Manual da eletrônica**. Disponível em: <https://www.manualdaeletronica.com.br/o-que-e-pwm-funcionamento-e-aplicacoes/>. Acesso em: 06 set. 2024.

BAUERMEISTER, Giovanni. Como usar Servo Motor com Arduino. **Fazedores**, 30 nov. 2018. Disponível em: <https://blog.fazedores.com/como-usar-servo-motor-com-arduino/>. Acesso em: 10 set. 2024.

BUZZER. **SunFounder**, 2022. Disponível em: <https://docs.sunfounder.com/projects/sf-components/en/latest/component_buzzer.html>. Acesso em: 10 set. 2024.

CASTRO, Leandro. Entendo o funcionamento do sensor inclinação (Giroscópio) com Arduino. **Arduino Ômega Blog**, 24 abril 2021. Disponível em: <https://blog.arduinoomega.com/5-passos-para-utilizar-o-sensor-de-inclinacao-giroscopio/>. Acesso em: 10 set. 2024.

CHAVE Táctil Push Button Com Capa. **Bitmarker**. Disponível em: <https://www.bitmaker.com.br/chave-tactil-push-button-com-capa->. Acesso em: 06 set. 2024.

CODEBENDER_CC. How to Use a Buzzer (or Piezo Speaker) - Arduino Tutorial. **Autodesk Instructables**, 2015. Disponível em: <https://www.instructables.com/How-to-use-a-Buzzer-Arduino-Tutorial/>. Acesso em: 10 set. 2024.

DAMIRCHI, Mohammad. Interfacing Passive Buzzer Module with Arduino. **Electro peak!**, 2020. Disponível em: <https://electropeak.com/learn/interfacing-passive-buzzer-module-with-arduino/>. Acesso em: 10 set. 2024.

DFROBOT. Ultrasonic Sensor Review: Comparing DFRobot URM09, HC-SR04, Devantech SFR02 & Maxbotix MB1040. **DFRobot**, 23 nov. 2023. Disponível em: <https://www.dfrobot.com/blog-13482.html>. Acesso em: 10 set. 2024.

DHT22 – Temperature and Humidity Sensor. **Components101**, 19 de abril de 2018. Disponível em: <https://components101.com/sensors/dht22-pinout-specs-datasheet>. Acesso em: 06 set. 2024.

DIYABLES DHT22 Temperature and Humidity Sensor Module for Arduino, ESP32, ESP8266, Raspberry Pi. **DIY Ables**, 2022. Disponível em:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

<https://diyables.io/products/dht22-temperature-and-humidity-sensor-module>. Acesso em: 10 set. 2024.

EXP. 02: Push Button Interfacing. **Gmostfabd**. 2024. Disponível em: <https://gmostofabd.github.io/8051-Push-Button/>. Acesso em: 06 set. 2024.

GRACHTEN, Eduardo. Servo motor - o que é e como funciona. **LinkedIn**, 09 de outubro de 2022. Disponível em: <https://www.linkedin.com/pulse/servo-motor-o-que-%C3%A9-e-como-funciona-eduardo-grachten/>. Acesso em: 06 set. 2024.

GAIN, Sudipto. DAY-2 Motion Sensor (HC-SR501). **LinkedIn**, 23 de fevereiro de 2024. Disponível em: <https://www.linkedin.com/pulse/day-2-motion-sensor-hc-sr501-shudipto-gain-tkpsc>. Acesso em: 06 set. 2024.

MATTEDE, Henrique. O que é servo motor e como funciona. **Manual da Eletrica**. Disponível em: <https://www.mundodaeletrica.com.br/o-que-e-servo-motor-e-como-funciona/>. Acesso em: 06 set. 2024.

HACER sensores de movimiento com PIR. **Proyecto Electronico**, 2017. Disponível em: <https://www.proyectoelectronico.com/alarmas/pir-re200b-biss001-lhi778-kc778b.html>. Acesso em: 10 set. 2024.

MÓDULO conversor de nível lógico 335V - Bidirecional 4 Canais. **Smartkits**. Disponível em: <https://www.smartkits.com.br/modulo-conversor-de-nivel-logico-3-3-5v-bidirecional?parceiro=9390&gad_source=1&gclid=CjwKCAjwuMC2BhA7EiwAmJKRrDg5_sgX4fBg996GcLVuDMi61lfligJzMP-OcYQyACbC8fZ08z4woxoCT4oQAvD_BwE>. Acesso em: 06 set. 2024

MÓDULO LDR (Resistor dependente da luz). **STA**, 2020. Disponível em: <https://www.sta-eletronica.com.br/artigos/arduinos/modulo-ldr-resistor-dependente-da-luz>. Acesso em: 10 set. 2024.

MÓDULO Relé 1 Canal Led Indicador para Arduino Pic 5V 10A. **Loja Robótica Educacional**. Disponível em: <https://loja.roboticaeducacional.art.br/modulo-rele-1-canal-led-indicador-para-arduino-pic-5v-10a>. Acesso em: 06 set. 2024.

VIEIRA, João. Acionando uma Lâmpada pela rede Ethernet. **Robocore**, 09 de julho de 2024. Disponível em: <https://www.robocore.net/tutoriais/acionando-uma-lampada-pela-rede-ethernet>. Acesso em: 06 set. 2024.

WHAT is PIR Motion Detector. **Mokolora**, 2022. Disponível em: <https://www.mokolora.com/wp-content/uploads/2022/03/What-is-PIR-Motion-Detector.webp>. Acesso em: 06 set. 2024.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com